



U N I V E R S I D A D
COMPLUTENSE
M A D R I D



Implementaciones de los temas 2 a 6

Máster en Ingeniería de Sistemas y Control

Asignatura: Visión por Computador

Curso: 2025/2026

Alumno: Javier Arambarri Calvo

24 de noviembre de 2025



Autoría

El autor de este documento es Javier Arambarri Calvo, alumno del Máster en Ingeniería de Sistemas y Control y matriculado por la Universidad Complutense de Madrid en el curso académico 2025-2026.

Para que así conste, se firma digitalmente este documento.

Imágenes utilizadas

Todas las imágenes utilizadas son libres, bien propias o extraídas de <https://pixabay.com/>.

Repositorio

El código de la implementación de los métodos expuestos se encuentra en el siguiente repositorio:
https://github.com/arambarricalvoj/vpc_misc_ucm-uned_25-26.



Índice

Tema 2: transformaciones del dominio y espaciales	6
Transformada de las Componentes Principales (PCA)	6
Algoritmo, implementación y codificación	7
Resultados	8
Transformada Discreta de Wavelets (DWT)	9
Algoritmo, implementación y codificación	10
Resultados	10
Transformada Discreta del Coseno (DCT)	12
Algoritmo, implementación y codificación	13
Resultados	13
Tema 3: color y correcciones radiométricas y geométricas	15
Suavizado: filtro gaussiano	15
Algoritmo, implementación y codificación	15
Resultados	16
Operaciones en el histograma de la imagen: ecualización	17
Algoritmo, implementación y codificación	17
Resultados	17
Registro automático entre imágenes	33
Algoritmo, implementación y codificación	33
Resultados	34
Tema 4: extracción de bordes, regiones y puntos de interés.	35
Extracción de bordes mediante operaciones en la primera derivada: Sobel	35
Algoritmo, implementación y codificación	35
Resultados	36
Extracción de puntos de interés mediante Harris	37
Algoritmo, implementación y codificación	37
Resultados	37
Extracción de bordes mediante operaciones en la segunda derivada: aproximación de Kim y Aggarwal	39
Algoritmo, implementación y codificación	39
Resultados	40
Tema 5: descripción de bordes y regiones.	41
Transformada de Hough	41
Algoritmo, implementación y codificación	43
Resultados	43
Contornos deformables	47
Algoritmo, implementación y codificación	48
Resultados	48
Tema 6: operaciones morfológicas.	50
Operaciones Morfológicas	50
Resultados	52

Índice de figuras

1.	Resultados en la imagen de Lena.	8
2.	Resultados en la imagen propia.	8
3.	Primera imagen original y su histograma.	18
4.	Segunda imagen original y su histograma.	18
5.	Primera imagen ecualizada uniformemente y su histograma.	19
6.	Segunda imagen ecualizada uniformemente y su histograma.	19
7.	Primera imagen ecualizada exponencialmente con $\alpha = 0,5$ y su histograma.	20
8.	Segunda imagen ecualizada exponencialmente con $\alpha = 0,5$ y su histograma.	20
9.	Primera imagen ecualizada exponencialmente con $\alpha = 2$ y su histograma.	21
10.	Segunda imagen ecualizada exponencialmente con $\alpha = 2$ y su histograma.	21
11.	Primera imagen ecualizada exponencialmente con $\alpha = 3,5$ y su histograma.	22
12.	Segunda imagen ecualizada exponencialmente con $\alpha = 3,5$ y su histograma.	22
13.	Primera imagen ecualizada <i>rayleigh</i> con $\sigma = 0,5$ y su histograma.	23
14.	Segunda imagen ecualizada <i>rayleigh</i> con $\sigma = 0,5$ y su histograma.	23
15.	Primera imagen ecualizada <i>rayleigh</i> con $\sigma = 2$ y su histograma.	24
16.	Segunda imagen ecualizada <i>rayleigh</i> con $\sigma = 2$ y su histograma.	24
17.	Primera imagen ecualizada <i>rayleigh</i> con $\sigma = 3,5$ y su histograma.	25
18.	Segunda imagen ecualizada <i>rayleigh</i> con $\sigma = 3,5$ y su histograma.	25
19.	Primera imagen ecualizada con $\gamma = 1$ y su histograma.	26
20.	Segunda imagen ecualizada con $\gamma = 1$ y su histograma.	26
21.	Primera imagen ecualizada con $\gamma = 2$ y su histograma.	27
22.	Segunda imagen ecualizada con $\gamma = 2$ y su histograma.	27
23.	Primera imagen ecualizada con $\gamma = 3$ y su histograma.	28
24.	Segunda imagen ecualizada con $\gamma = 3$ y su histograma.	28
25.	Primera imagen ecualizada logarítmicamente con $\alpha = 1$ y su histograma.	29
26.	Segunda imagen ecualizada logarítmicamente con $\alpha = 1$ y su histograma.	29
27.	Primera imagen ecualizada logarítmicamente con $\alpha = 2$ y su histograma.	30
28.	Segunda imagen ecualizada logarítmicamente con $\alpha = 2$ y su histograma.	30
29.	Primera imagen ecualizada logarítmicamente con $\alpha = 3$ y su histograma.	31
30.	Segunda imagen ecualizada logarítmicamente con $\alpha = 3$ y su histograma.	31
31.	Resultado del registro automático entre imágenes.	34
32.	Primera imagen original de la que se han extraído las rectas.	43
33.	Segunda imagen original de la que se han extraído las circunferencias.	43
34.	Imagen original sobre la que se han aplicado las operaciones morfológicas.	52

Índice de cuadros

1.	Resultados en la imagen de Lena.	8
2.	Resultados en la imagen propia.	8
3.	Resultados de la compresión de imágenes mediante <i>wavelets</i>	11
4.	Resultados en la imagen de Lena.	11
5.	Resultados en la imagen propia.	11
6.	Resultados en la imagen de Lena.	13
7.	Resultados en la imagen propia.	13
8.	Resultados de la compresión de imágenes mediante <i>DCT</i>	14
9.	Resultados del filtro gaussiano para distintos valores de σ y tamaño de filtro k	16
10.	Funciones de ecualización más comunes.	32
11.	Resultados de la implementación manual del operador Sobel.	36
12.	Resultados de la implementación <i>OpenCV</i> del operador Sobel.	36
13.	Resultados del detector de Harris.	38
14.	Filtros discretos de aproximación de la Laplaciana y su efecto en la detección de bordes.	39
15.	Resultados de la aproximación de Kim y Aggarwal para la extracción de bordes para distintos valores de los parámetros.	40
16.	Resultados de la transformada de Hough en la primera imagen.	45
17.	Resultados de la transformada de Hough en la segunda imagen.	46
18.	Resultados de la transformada de Hough en la segunda imagen.	49
19.	Resultados de la dilatación.	53
20.	Resultados de la erosión.	54
21.	Resultados de la apertura.	55
22.	Resultados del cierre.	56
23.	Resultados de la operación de bordes.	57
24.	Resultados de la operación de vaciado sobre los bordes extraídos en rectángulo y círculo con un tamaño de 7 y 17.	58
25.	Resultados de la operación de llenado con distintos tamaños y formas del elemento estructurante y valor de umbral de binarización.	59
26.	Resultados del engrosado.	60
27.	Resultados del adelgazamiento.	61

Tema 2: transformaciones del dominio y espaciales

Transformada de las Componentes Principales (PCA)

La Transformada de Componentes Principales (PCA) es una técnica de **reducción de dimensionalidad** ampliamente utilizada en el análisis de datos. En el ámbito del tratamiento de imágenes, su objetivo principal es la **compresión** y representación eficiente de la información visual.

Este método se fundamenta en transformaciones derivadas del álgebra lineal que permiten **identificar las características o componentes principales** de un conjunto de datos, ya sean imágenes, series temporales u otros tipos de información. Una vez determinadas estas características esenciales, **los datos originales se proyectan** sobre dichas direcciones principales, lo que permite conservar la información más relevante y descartar la redundante o poco significativa.

Desde un punto de vista matemático, el proceso se basa en el cálculo de **autovectores** y **autovalores** de la **matriz de covarianza** de los datos. Si se considera una imagen como una matriz de píxeles de dimensiones x e y , PCA permite encontrar las direcciones de variación más significativas (autovectores) y medir su importancia relativa (autovalores). Los componentes principales asociados a los mayores autovalores se seleccionan para proyectar el conjunto original de datos, simplificando así la representación sin perder la esencia de la información inicial. Esos autovalores corresponden con la varianza de cada característica o columna, y cuanto más varianza capte un componente, mayor información relevante mantiene. Por tanto, **si queremos conservar el 95 % de la información destacada de una imagen, PCA proyecta la información original sobre las componentes o las direcciones que representan el 95 % de la varianza**.

La proyección se refiere a calcular la componente de cada punto o elemento de datos a lo largo de estas direcciones principales, esto es, transformar los datos desde su espacio original hacia un nuevo sistema de coordenadas definido por esos autovectores. En términos prácticos, se multiplica la matriz de datos original por la matriz cuyos vectores columnas son los autovectores seleccionados.

En resumen, la Transformada de Componentes Principales **transforma los datos originales en un nuevo conjunto de variables no correlacionadas —los componentes principales—, ordenadas según la cantidad de varianza que explican del conjunto original**. Así, se logra **reducir la dimensionalidad manteniendo la mayor parte de la variabilidad o información contenida en los datos**. Por ejemplo, si una imagen está representada por cientos de miles de píxeles, proyectarla sobre los principales autovectores con mayor varianza permite reducir ese número a unas pocas variables que sintetizan la información más significativa de la imagen, facilitando su análisis y compresión.

Algoritmo, implementación y codificación

Se ha utilizado la referencia [1], que describe con claridad el proceso del método *PCA*, para obtener los siguientes pasos para comprimir una imagen en escala de grises:

1. **Leer la imagen, cargarla en blanco y negro, convertirla a representación de 64 bits para aumentar la precisión de las transformaciones y normalizarla en el rango [0.0, 1.0].**

Llamamos $X_{64F} \in \mathbb{R}^{H \times W}$ a la matriz de datos normalizada en coma flotante (64 bits):
$$X_{64F} = \frac{\text{double}(X_{8U})}{255,0}.$$

2. **Calcular la matriz de covarianza.** Cada columna de la imagen se considera una característica W y cada fila una observación H , por lo que la varianza de cada columna o característica mide cómo varían sus observaciones.

Cada elemento de esta matriz representa la covarianza entre dos variables, es decir, la relación lineal entre ellas, reflejando si tienden a aumentar o disminuir juntas (covarianza positiva), o si una crece cuando la otra decrece (covarianza negativa). En la diagonal de la matriz se encuentran las varianzas de cada variable, característica o columna.

A cada columna de $X_{64F} \in \mathbb{R}^{H \times W}$ se le resta la media y se obtiene la matriz centrada $X_c \in \mathbb{R}^{H \times W}$: $X_c = X_{64F} - \mathbf{1}_H \bar{\mathbf{x}}^T$, donde $\bar{\mathbf{x}} = (\bar{x}_1, \bar{x}_2, \dots, \bar{x}_W)^T$ es el vector fila de medias de cada característica y $\mathbf{1}_H \in \mathbb{R}^{H \times 1}$ es un vector columna de unos.

La matriz de covarianza $C \in \mathbb{R}^{W \times W}$ se calcula como: $C = \frac{1}{H-1} X_c^T X_c$.

Cada elemento c_{ij} de C es la covarianza entre las variables i y j y se define como:
$$c_{ij} = \frac{1}{H-1} \sum_{k=1}^H (X_{ki} - \bar{x}_i)(X_{kj} - \bar{x}_j).$$
 Aquí, X_{ki} es el valor de la observación k para la característica i , y $\bar{x}_i$ la media de esa característica.

3. **Cálculo de los componentes principales.** Se calculan los autovectores y autovalores de la matriz de covarianza: $C\mathbf{v} = \lambda\mathbf{v}$, donde $\mathbf{v}$ es un autovector (dirección principal) y λ el autovalor (magnitud de la varianza).

4. **Selección de componentes principales.** Se ordenan los autovalores de mayor a menor $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_W$, y se seleccionan los primeros k autovectores correspondientes a los autovalores que acumulan el porcentaje deseado de varianza a mantener:

$$k = \min \left\{ m : \frac{\sum_{i=1}^m \lambda_i}{\sum_{j=1}^W \lambda_j} \geq \text{var_ratio} \right\}.$$

5. **Proyección de los datos sobre los autovectores seleccionados** $W_k \in \mathbb{R}^{W \times k}$ para obtener la representación comprimida $Z \in \mathbb{R}^{H \times k}$: $Z = X_c W_k$.

Finalmente, la reconstrucción aproximada o comprimida de la imagen se obtiene mediante $\hat{X} = ZW_k^T + \mathbf{1}\bar{\mathbf{x}}^T$, donde $\mathbf{1}$ es un vector de unos para reintroducir la media en cada característica.

La codificación e implementación del método se ha realizado en el lenguaje **C++** con la ayuda de las librerías *OpenCV*[2] y *Eigen* [3].

Resultados

Se han realizado pruebas con dos imágenes: la imagen de Lena y una imagen propia. Para ambas se han realizado tres compresiones: manteniendo el 99 % de la varianza, el 95 % y el 80 %. En las figuras 1 y 2 se observan los resultados visuales, mientras que en las tablas 1 y 2 los resultados numéricos.

Analizando las figuras y los cuadros se obtiene que los últimos valores de varianza son clave para detallar las imágenes y por tanto, al reducir su tamaño pierden precisión en la representación. Sin embargo, en los casos en los que únicamente se reduce la varianza en un 1%, la pérdida de información es prácticamente imperceptible al ojo humano.

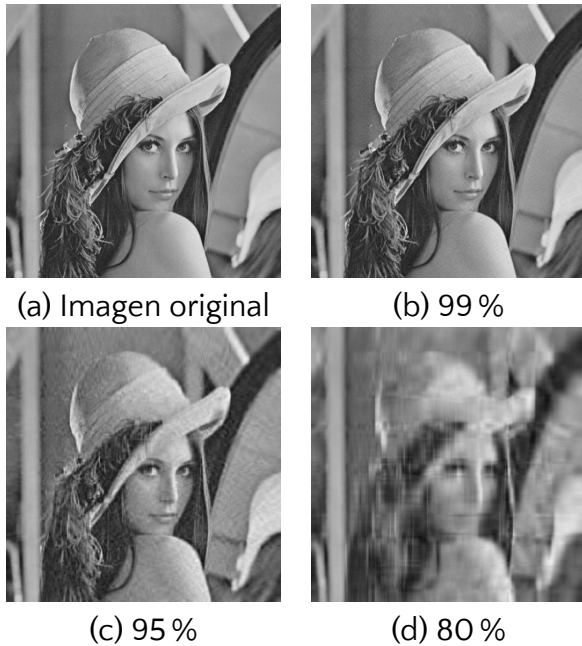


Figura 1: Resultados en la imagen de Lena.

Lena	Tamaño	k	Reducción
Original	167.6 kB	-	-
99	162.3 kB	101	3 %
95	154 kB	39	8 %
80	135 kB	11	20 %

Cuadro 1: Resultados en la imagen de Lena.

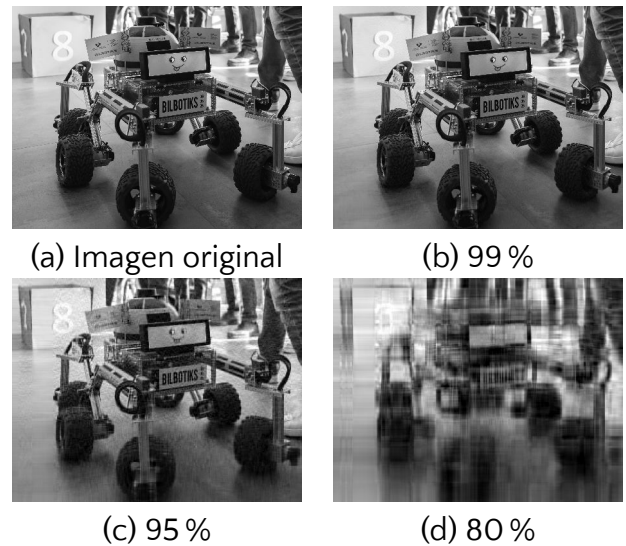


Figura 2: Resultados en la imagen propia.

Propia	Tamaño	k	Reducción
Original	1.9 MB	-	-
99	1.8 MB	200	5 %
95	1.7 MB	51	10 %
80	135 kB	10	26 %

Cuadro 2: Resultados en la imagen propia.

Nota: para comprimir la imagen propia el algoritmo implementado necesitaba alrededor de 8 minutos. Esto se debe a la elevada cantidad de píxeles que forman la imagen y al elevado número de operaciones entre matrices a realizar. Es necesario optimizar el algoritmo mediante diferentes técnicas fuera del objeto de la asignatura, o mediante aceleración por GPU y CUDA.

Transformada Discreta de Wavelets (DWT)

La Transformada Discreta de Wavelets (DWT) es una técnica que permite **descomponer una señal o imagen en diferentes niveles de detalle**. En el ámbito del tratamiento de imágenes, su objetivo principal es la **compresión con pérdida controlada**, separando la información de baja frecuencia (estructura global) de la información de alta frecuencia (detalles y bordes).

A diferencia de la Transformada de Componentes Principales, que se basa en el álgebra lineal y la covarianza de los datos, la DWT se fundamenta en el uso de **filtros paso bajo y paso alto** aplicados de manera jerárquica. La wavelet más simple es la **Haar**, que se define mediante operaciones de **promedio y diferencia** sobre pares de valores consecutivos.

En la **transformada Haar 1D**, aplicada sobre un vector de datos (por ejemplo, una fila de píxeles), para cada par de elementos (x_{2i}, x_{2i+1}) , donde x_{2i} es el elemento en posición par de la secuencia y x_{2i+1} el inmediatamente siguiente en posición impar, se calculan:

$$a_i = \frac{x_{2i} + x_{2i+1}}{\sqrt{2}} \quad (\text{coeficiente de aproximación, paso bajo})$$

$$d_i = \frac{x_{2i} - x_{2i+1}}{\sqrt{2}} \quad (\text{coeficiente de detalle, paso alto})$$

El resultado es un nuevo vector formado por los coeficientes de aproximación a_i y los coeficientes de detalle d_i .

En la **transformada Haar 2D**, aplicada sobre una matriz de datos (una imagen), se procede en dos pasos:

1. Se aplica la transformada Haar 1D a cada fila de la imagen.
2. Se aplica la transformada Haar 1D a cada columna del resultado anterior.

De esta forma, la imagen se descompone en cuatro sub-bandas:

- **LL**: paso bajo en filas y columnas, contiene la **aproximación** o baja frecuencia global.
- **LH**: paso bajo en filas y paso alto en columnas, contiene los **detalles horizontales**.
- **HL**: paso alto en filas y paso bajo en columnas, contiene los **detalles verticales**.
- **HH**: paso alto en filas y columnas, contiene los **detalles diagonales**.

Visualmente, la matriz de coeficientes resultante se organiza en cuadrantes:

$$\begin{bmatrix} LL & LH \\ HL & HH \end{bmatrix}$$

La **transformada inversa Haar (IDWT)** reconstruye la señal o imagen original a partir de los coeficientes de aproximación y detalle. En el caso 1D, para cada par (a_i, d_i) se recuperan los valores originales:

$$x_{2i} = \frac{a_i + d_i}{\sqrt{2}}, \quad x_{2i+1} = \frac{a_i - d_i}{\sqrt{2}}$$

En el caso 2D, se aplica primero la inversa en columnas y después en filas, combinando nuevamente promedios y diferencias para obtener la matriz original.

La compresión se logra mediante la **umbralización de coeficientes**. Los coeficientes de alta frecuencia con valores pequeños se eliminan (se ponen a cero), ya que representan detalles poco relevantes o ruido. Matemáticamente, el umbral se aplica como:

$$c' = \begin{cases} 0 & \text{si } |c| < T \\ c & \text{si } |c| \geq T \end{cases}$$

Algoritmo, implementación y codificación

1. **Leer la imagen, cargarla en blanco y negro, convertirla a representación de 64 bits para aumentar la precisión de las transformaciones y normalizarla en el rango [0.0, 1.0].** Llamamos $X_{64F} \in \mathbb{R}^{H \times W}$ a la matriz de datos normalizada en coma flotante (64 bits): $X_{64F} = \frac{\text{double}(X_{8U})}{255,0}$.
2. **Aplicar la transformada Haar 2D.** Primero se aplica la transformada Haar 1D a cada fila de la matriz, y posteriormente a cada columna, obteniendo la matriz de coeficientes C que contiene las cuatro sub-bandas LL, LH, HL, HH .
3. **Umbralización de coeficientes.** Se aplica un umbral T a los coeficientes de alta frecuencia (LH, HL, HH), eliminando aquellos cuya magnitud es menor que T . Esto reduce la cantidad de información almacenada y logra la compresión.
4. **Reconstrucción de la imagen.** Se aplica la transformada inversa Haar 2D sobre la matriz de coeficientes umbralizados para obtener la imagen comprimida $\hat{X}$.
5. **Escalado y guardado.** La imagen reconstruida se recorta al rango [0,1], se convierte a formato de 8 bits y se guarda en la carpeta de salida.

La codificación e implementación del método se ha realizado en el lenguaje **C++** con la ayuda de la librería *OpenCV*.

Resultados

Se han realizado pruebas con las mismas imágenes que en la compresión mediante *PCA*. En ambos casos se han aplicado diferentes valores de umbral T para observar el efecto en la compresión.

En la tabla 3 se muestran los resultados visuales, mientras que en las tablas 4 y 5 se presentan los resultados numéricos.

En comparación con el método anterior, se observa que se consigue una mayor compresión con menor pérdida de resolución visual, es decir, gráficamente la imagen sigue prácticamente intacta, pero ocupa un considerable menor tamaño. Esto tiene sentido ya que la compresión se logra eliminando coeficientes pequeños en las bandas de alta frecuencia, preservando las estructuras grandes y eliminando detalles poco relevantes.

Por otra parte, si se realizan pruebas con valores altos de umbral se obtendrían regiones negras en la imagen, que representan la pérdida de información. Por tanto, para lograr los resultados presentados hay que probar diferentes umbrales hasta encontrar el óptimo.



Cuadro 3: Resultados de la compresión de imágenes mediante *wavelets*.

Lena	Tamaño	Reducción
Original	167.6 kB	-
$T = 0,02$	147.4 kB	12 %
$T = 0,2$	117 kB	30 %

Cuadro 4: Resultados en la imagen de Lena.

Propia	Tamaño	Reducción
Original	1.9 MB	-
$T = 0,02$	1.6 MB	15 %
$T = 0,05$	1.4 MB	26 %

Cuadro 5: Resultados en la imagen propia.

Transformada Discreta del Coseno (*DCT*)

Es una técnica que permite **representar una señal o imagen como una combinación de funciones coseno de diferentes frecuencias**. En el ámbito del tratamiento de imágenes, su objetivo principal es la **compresión con pérdida controlada**, concentrando la mayor parte de la energía en los coeficientes de baja frecuencia y permitiendo eliminar o cuantizar los coeficientes de alta frecuencia.

A diferencia de la Transformada de Componentes Principales, que se basa en la covarianza de los datos, o de la Transformada de Wavelets, que utiliza filtros paso bajo y paso alto, la *DCT* se fundamenta en la expansión de la señal en una base de cosenos ortogonales.

En la **transformada DCT 1D**, aplicada sobre un vector de datos x_n de longitud N , los coeficientes X_k se calculan como:

$$X_k = \alpha(k) \sum_{n=0}^{N-1} x_n \cos \left[\frac{\pi}{N} \left(n + \frac{1}{2} \right) k \right], \quad k = 0, 1, \dots, N-1$$

donde el factor de normalización es:

$$\alpha(k) = \begin{cases} \sqrt{\frac{1}{N}} & \text{si } k = 0 \\ \sqrt{\frac{2}{N}} & \text{si } k > 0 \end{cases}$$

En la **transformada DCT 2D**, aplicada sobre una matriz de datos (una imagen), se procede de la siguiente manera:

1. Se aplica la DCT 1D a cada fila de la imagen.
2. Se aplica la DCT 1D a cada columna del resultado anterior.

El resultado es una matriz de coeficientes donde:

- Los coeficientes cercanos a la esquina superior izquierda representan la **baja frecuencia** (estructura global).
- Los coeficientes alejados representan la **alta frecuencia** (detalles y bordes).

La **transformada inversa DCT (IDCT)** reconstruye la señal o imagen original a partir de los coeficientes, aplicando la misma fórmula pero con cosenos en fase inversa:

$$x_n = \sum_{k=0}^{N-1} \alpha(k) X_k \cos \left[\frac{\pi}{N} \left(n + \frac{1}{2} \right) k \right], \quad n = 0, 1, \dots, N-1$$

La compresión se logra mediante la **umbralización de coeficientes**. Los coeficientes de alta frecuencia con valores pequeños se eliminan (se ponen a cero), ya que representan detalles poco relevantes o ruido. Matemáticamente, el umbral se aplica como:

$$c' = \begin{cases} 0 & \text{si } |c| < T \\ c & \text{si } |c| \geq T \end{cases}$$

Algoritmo, implementación y codificación

1. **Leer la imagen, cargarla en blanco y negro, convertirla a representación de 64 bits en coma flotante y normalizarla en el rango [0.0, 1.0].** Llamamos $X_{64F} \in \mathbb{R}^{H \times W}$ a la matriz de datos normalizada: $X_{64F} = \frac{\text{float}(X_{8U})}{255,0}$.
2. **Aplicar la transformada *DCT* 2D.** Primero se aplica la *DCT* 1D a cada fila de la matriz, y posteriormente a cada columna, obteniendo la matriz de coeficientes C . En la implementación se han utilizado las funciones `cv::dct` y `cv::idct` de la librería *OpenCV*.
3. **Umbralización de coeficientes.** Se aplica un umbral T a los coeficientes de alta frecuencia, eliminando aquellos cuya magnitud es menor que T . Esto reduce la cantidad de información almacenada y logra la compresión.
4. **Reconstrucción de la imagen.** Se aplica la transformada inversa *DCT* 2D sobre la matriz de coeficientes umbralizados para obtener la imagen comprimida $\hat{X}$.
5. **Escalado y guardado.** La imagen reconstruida se normaliza o recorta al rango [0,1], se convierte a formato de 8 bits y se guarda en la carpeta de salida.

Resultados

Se han realizado pruebas con las mismas imágenes que en la compresión mediante *PCA* y *Wavelets*. En ambos casos se han aplicado diferentes valores de umbral T para observar el efecto en la compresión.

En la tabla 8 se muestran los resultados visuales, mientras que en las tablas 6 y 7 se presentan los resultados numéricos.

En comparación con los métodos anteriores, la *DCT* consigue una compresión muy eficiente, especialmente en imágenes con regiones suaves, ya que la energía se concentra en pocos coeficientes de baja frecuencia. Sin embargo, si se aplican umbrales demasiado altos, aparecen artefactos visibles en forma de bloques o pérdida de nitidez provocados por la eliminación de coeficientes de alta frecuencia.

Lena	Tamaño	Reducción
Original	167.6 kB	-
$T = 0,15$	156 kB	7 %
$T = 0,35$	143.4 kB	14 %

Cuadro 6: Resultados en la imagen de Lena.

Propia	Tamaño	Reducción
Original	1.9 MB	-
$T = 0,15$	1.9 MB	0 %
$T = 0,35$	1.7 MB	10 %

Cuadro 7: Resultados en la imagen propia.



Cuadro 8: Resultados de la compresión de imágenes mediante *DCT*.

Tema 3: color y correcciones radiométricas y geométricas

Suavizado: filtro gaussiano

El filtro gaussiano es una técnica de suavizado. Su objetivo principal es **reducir el ruido y las variaciones de alta frecuencia**, preservando las estructuras más relevantes de la imagen. Para ello, se utiliza una función gaussiana como máscara, filtro o *kernel* de convolución, que asigna mayor peso a los píxeles cercanos al centro y menor a los más alejados. Es decir, **los píxeles a los que se les aplica la máscara seguirán una distribución gaussiana tras el suavizado**.

El núcleo gaussiano de desviación estándar σ se define como:

$$G_{\sigma}(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

donde (x, y) son las coordenadas relativas al centro del *kernel*. El tamaño del kernel, k , suele elegirse como un número impar cercano a $6 \cdot \sigma + 1$, de forma que se abarque la mayor parte de la distribución gaussiana.

El parámetro más importante del filtro gaussiano es la **desviación estándar**, σ , que controla el grado de suavizado aplicado a la imagen:

- Valores pequeños de σ (por ejemplo, $\sigma \leq 1$), el gaussiano es estrecho y concentra la mayor parte del peso en el píxel central. El suavizado es leve y se eliminan únicamente ruidos de alta frecuencia muy finos.
- Valores grandes de σ (por ejemplo, $\sigma \geq 2$), el gaussiano se ensancha, distribuyendo el peso entre píxeles más lejanos. El suavizado es fuerte, se difuminan los bordes y se pierden detalles de la imagen.

La regla $6 \cdot \sigma + 1$ para seleccionar el tamaño del filtro viene de la estadística matemática, de la que se sabe que el 99,7 % de la masa de la gaussiana estará en el intervalo $[-3\sigma, +3\sigma]$. La adición de una unidad permite que k sea impar para garantizar la presencia de un píxel central.

El resultado del suavizado se obtiene mediante la convolución de la imagen original $f(x, y)$ con el núcleo gaussiano:

$$g(x, y) = f(x, y) * G_{\sigma}(x, y)$$

Algoritmo, implementación y codificación

1. Se define el **tamaño de la máscara** $k \times k$ y el valor de la **desviación estándar** σ .
2. Se construye la máscara gaussiana $G_{\sigma}(x, y)$ **evaluando la función gaussiana en cada posición** (x, y) relativa al centro del *kernel*. Se normaliza para que la suma de todos sus valores sea 1.
3. Se aplica la **convolución** de la imagen original $f(x, y)$ con la máscara gaussiana $G_{\sigma}(x, y)$, obteniendo la imagen suavizada $g(x, y)$. En la práctica, esto se implementa multiplicando cada vecindad de píxeles por los coeficientes del kernel y sumando los resultados.

- Finalmente, se guarda la imagen suavizada, que presentará menos ruido y transiciones más suaves en las intensidades.

La codificación e implementación del método se ha realizado en el lenguaje **C++** con la ayuda de la librería *OpenCV*. No obstante, con el objetivo de aprender el procedimiento se ha optado por implementar manualmente tanto el filtro gaussiano como la operación de convolución en lugar de utilizar las funciones *GaussianBlur()* y *filter2d()*. La función de filtro gaussiano aquí creada se utiliza en la implementación de la aproximación de Kim y Aggarwal del tema 4 .

Resultados

Se ha aplicado el filtro gaussiano con los valores de $\sigma = 0,5, 1,5, 5$ y $k = 3$ y al valor correspondiente calculado según σ a la imagen de Lena. Los resultados se pueden ver en la tabla 9. Se observa claramente la importancia de que el tamaño del filtro sea acorde a la desviación típica.



Cuadro 9: Resultados del filtro gaussiano para distintos valores de σ y tamaño de filtro k .

Operaciones en el histograma de la imagen: ecualización

El histograma de una imagen es una **representación gráfica de los diferentes tonos de gris que contiene y su distribución**. Este gráfico tiene dos dimensiones: en el eje horizontal X se muestran los distintos valores de la escala de grises de la imagen y en el eje vertical Y se muestra la cantidad de píxeles que contiene cada tono. Si la imagen es de 8 bits por píxel la escala de grises podrá tomar 256 valores diferentes en el rango $[0, 255]$.

Son especialmente útiles para comprender la intensidad del brillo o luminosidad que tiene una imagen y por tanto, **cómo se distribuye esta luminosidad** a lo largo de la misma. Una de las operaciones que se puede aplicar sobre un histograma es la **ecualización o aumento de contraste**, que tiene como objetivo redistribuir los niveles de gris a lo largo de la imagen para que se repartan de manera más uniforme o siguiendo alguna distribución específica.

Algoritmo, implementación y codificación

1. El primer paso es **obtener la función de probabilidad**, $p(r_k)$, de que un determinado píxel tome un valor de intensidad r_k . Se consigue fácilmente normalizando el histograma, donde n_k es la cantidad de píxeles con la intensidad r_k y MN el número total de píxeles de la imagen: $p(r_k) = \frac{n_k}{MN}$.
2. A continuación se **calcula la función de distribución acumulada** (FDA o *CDF* por sus siglas en inglés), $P(r_k)$, que es la suma acumulada de las probabilidades hasta r_k :

$$P_r(r_k) = \sum_{j=0}^k p(r_j), \quad k = 0, 1, \dots, L - 1.$$

3. Por último, se aplica una **función de ecualización para que los niveles de gris de la imagen sigan una determinada distribución**. Las más comunes se recogen en la tabla 10, donde:
 - L es la cantidad de niveles de gris.
 - s es el valor de intensidad reescalado o transformado.
 - $\exp(x)$ es la función exponencial, es decir, e^x .

La codificación e implementación del método se ha realizado manualmente en el lenguaje **C++** con la ayuda de la librería *OpenCV*.

Resultados

Se han implementado las funciones del cuadro 10 a dos imágenes diferentes. En aquellas distribuciones con parámetros, se han probado tres parámetros diferentes que permitan observar su comportamiento. Se presentan las imágenes originales junto con su histograma, 3 y 4, y las imágenes ecualizadas junto con su histograma, figuras 5 - 30.

En función de las características de la imagen original será necesario utilizar una función u otra. En este caso, para la primera imagen se observa que la ecualización uniforme es muy brusca generando borrosidades, y la función que mejor se le ajusta es la exponencial ya que contiene mucha luminosidad y esta función aumenta el contraste en intensidades bajas (cuadro 10).

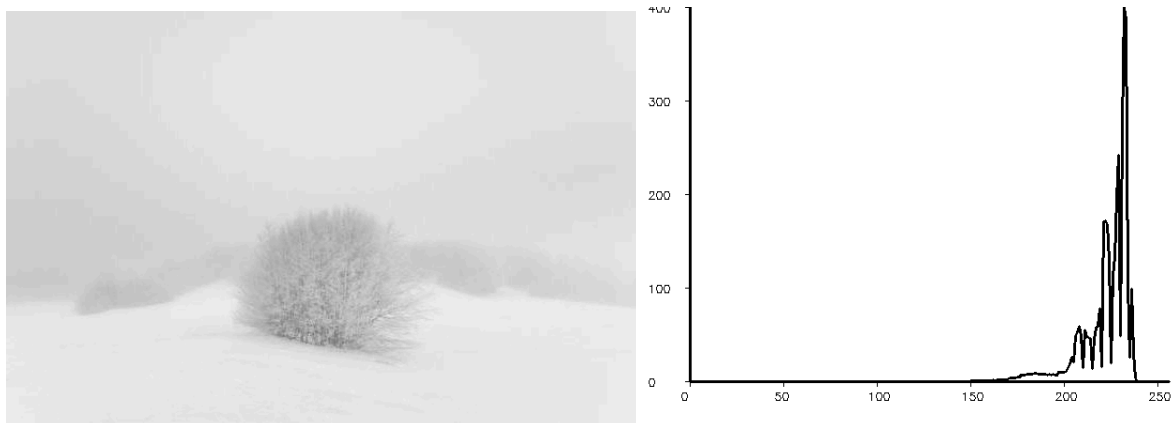


Figura 3: Primera imagen original y su histograma.

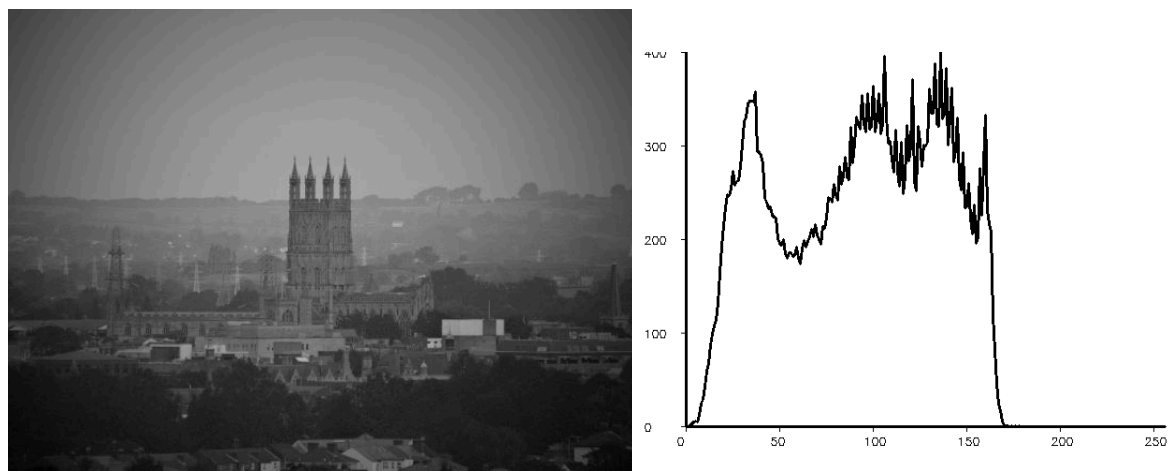


Figura 4: Segunda imagen original y su histograma.

- Ecualización uniforme

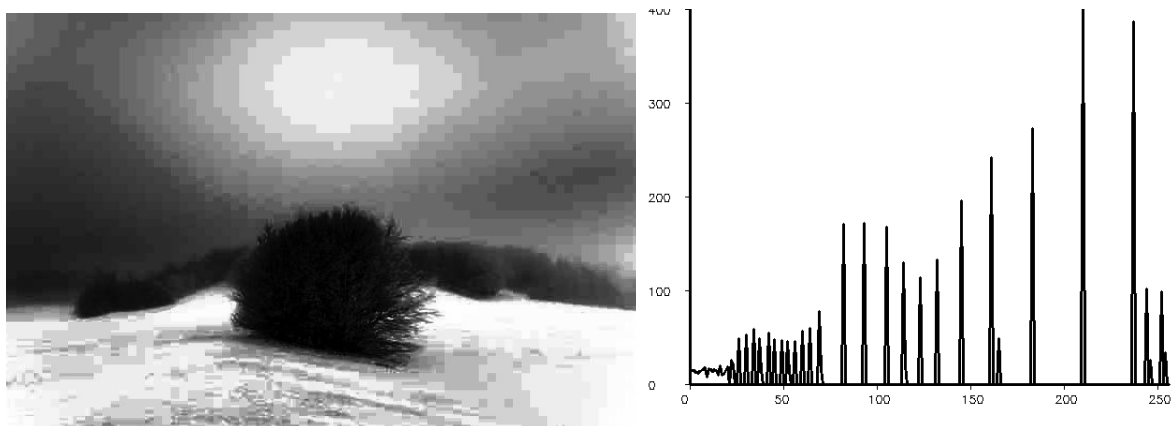


Figura 5: Primera imagen ecualizada uniformemente y su histograma.

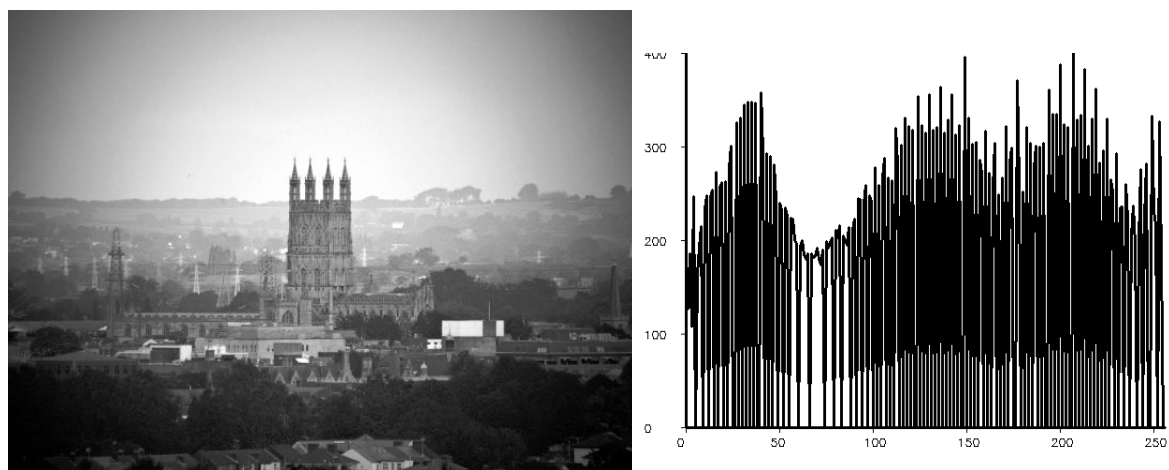


Figura 6: Segunda imagen ecualizada uniformemente y su histograma.

- Ecualización exponencial con $\alpha = 0,5, 2, 3, 5$.

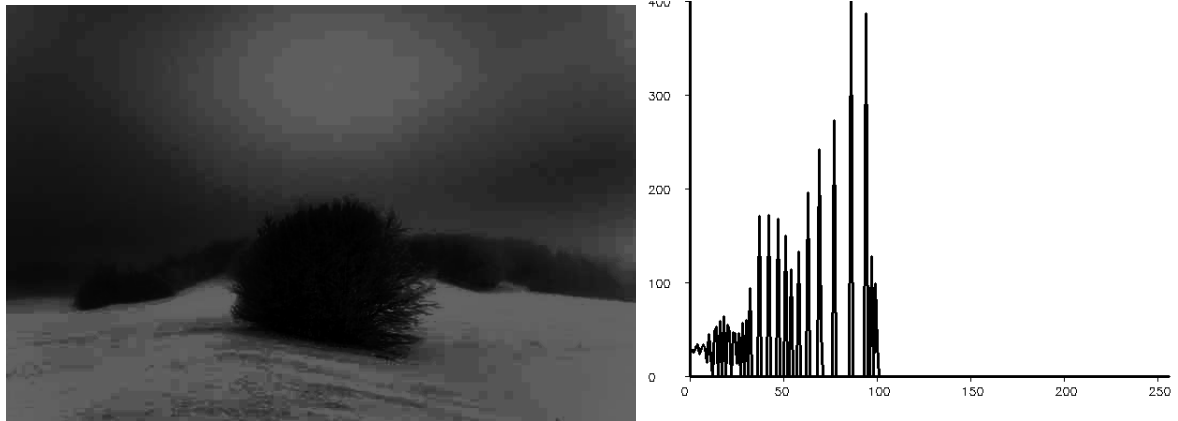


Figura 7: Primera imagen ecualizada exponencialmente con $\alpha = 0,5$ y su histograma.

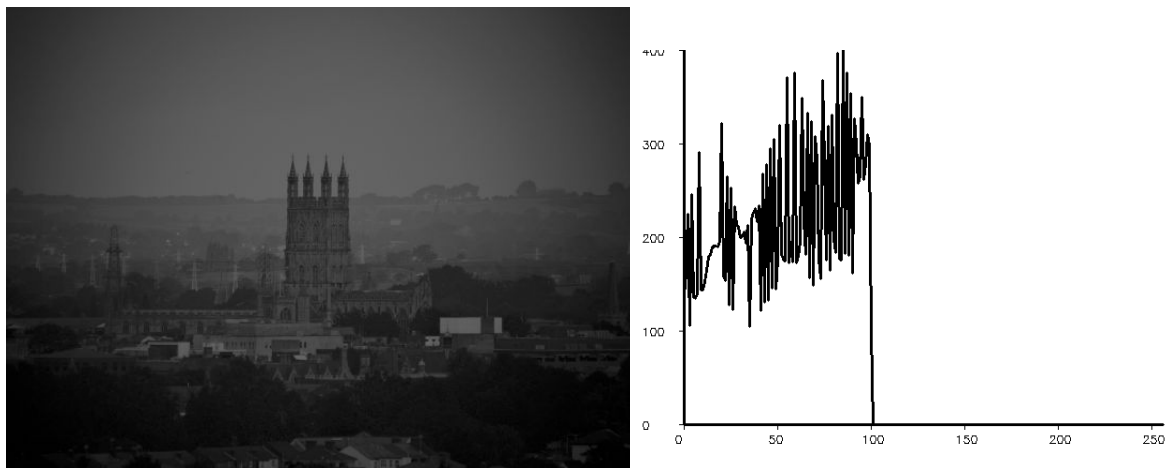


Figura 8: Segunda imagen ecualizada exponencialmente con $\alpha = 0,5$ y su histograma.

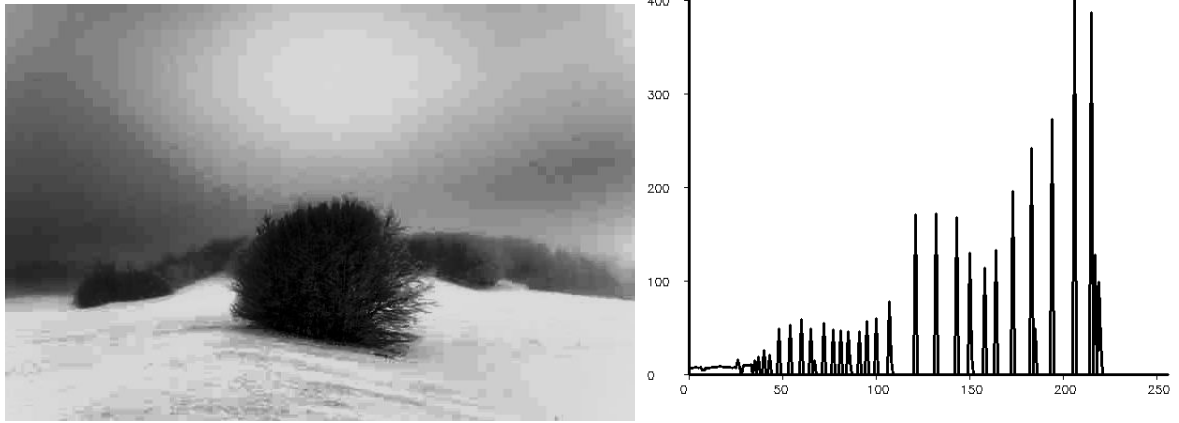


Figura 9: Primera imagen ecualizada exponencialmente con $\alpha = 2$ y su histograma.

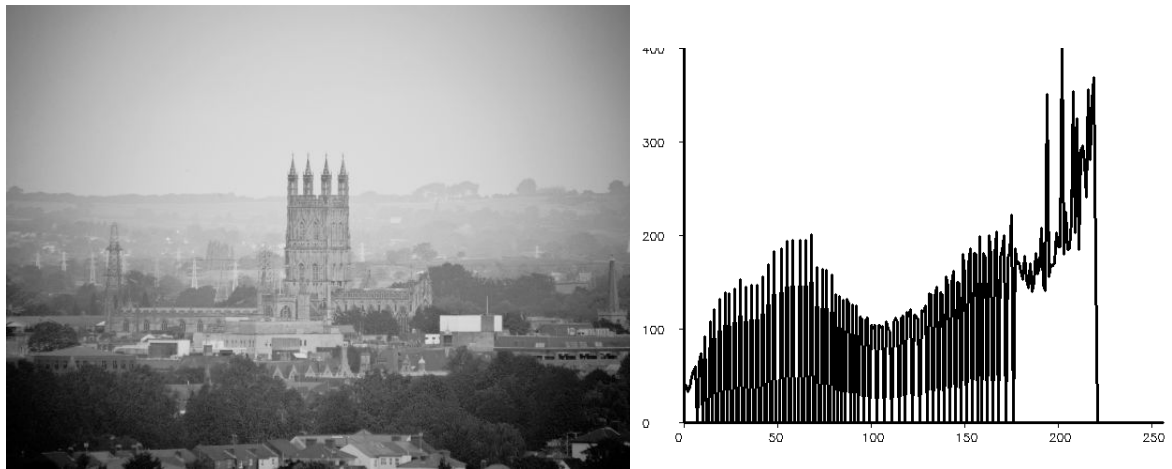


Figura 10: Segunda imagen ecualizada exponencialmente con $\alpha = 2$ y su histograma.

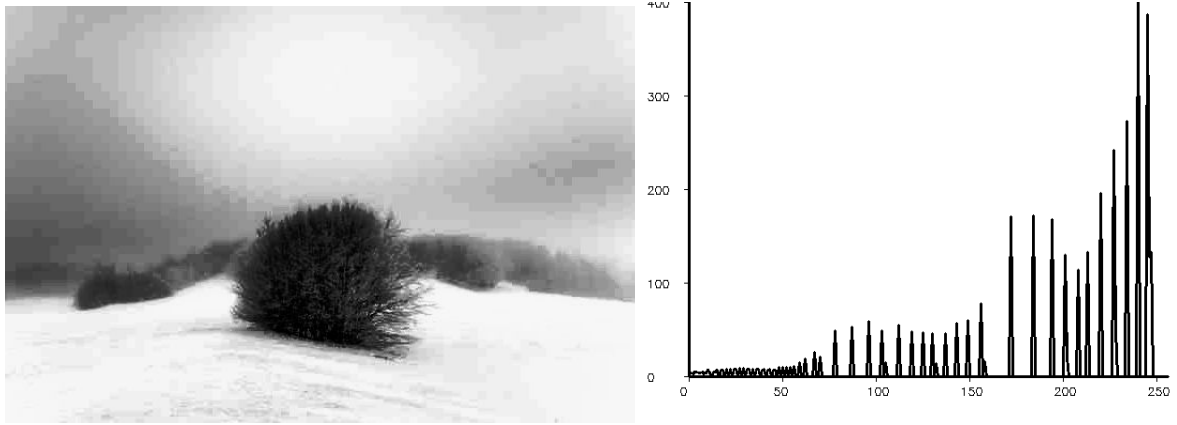


Figura 11: Primera imagen ecualizada exponencialmente con $\alpha = 3,5$ y su histograma.

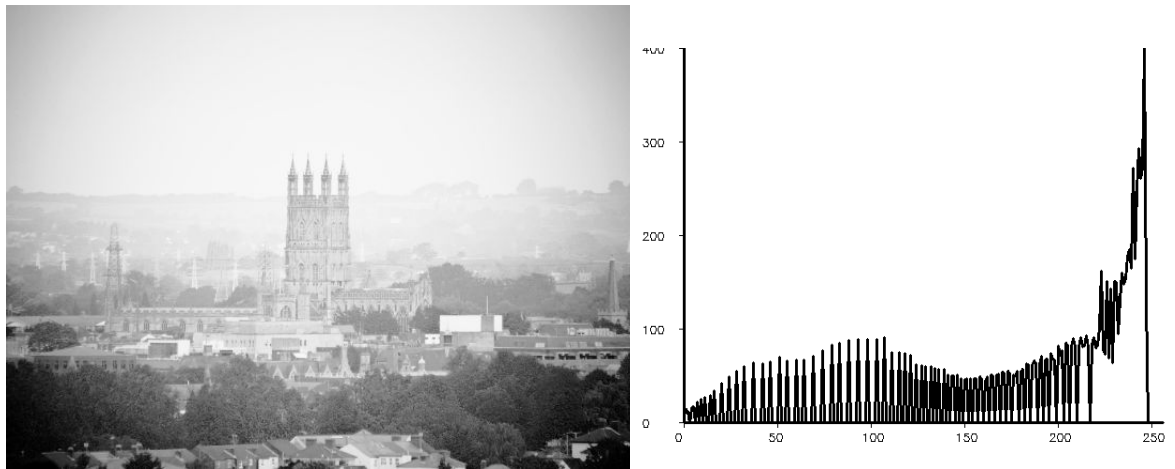


Figura 12: Segunda imagen ecualizada exponencialmente con $\alpha = 3,5$ y su histograma.

- Ecualización *rayleigh* con $\sigma = 0,5, 2, 3, 5$.

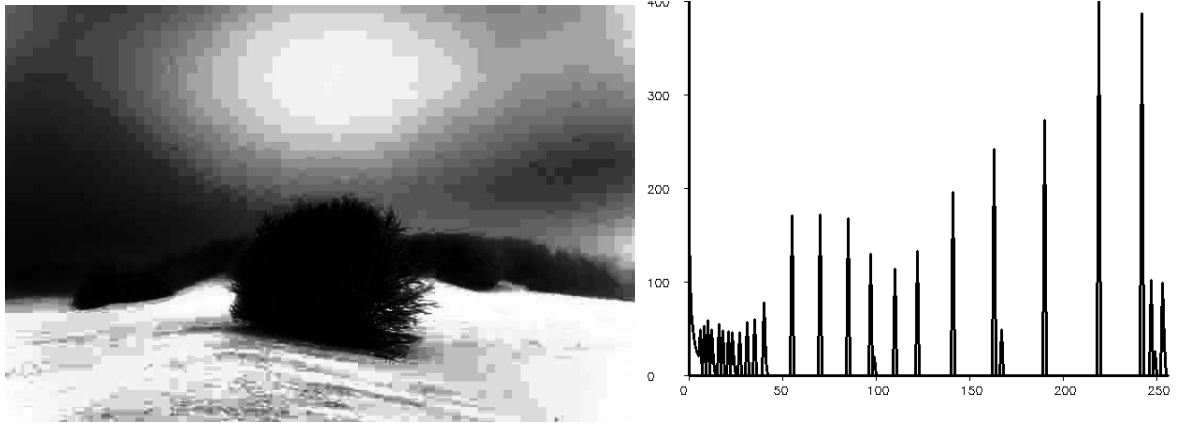


Figura 13: Primera imagen ecualizada *rayleigh* con $\sigma = 0,5$ y su histograma.

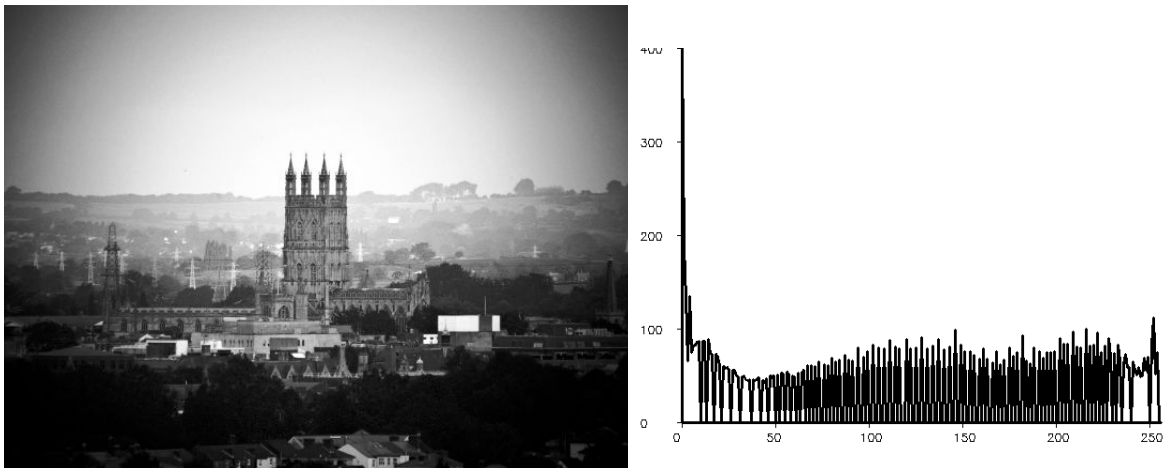


Figura 14: Segunda imagen ecualizada *rayleigh* con $\sigma = 0,5$ y su histograma.

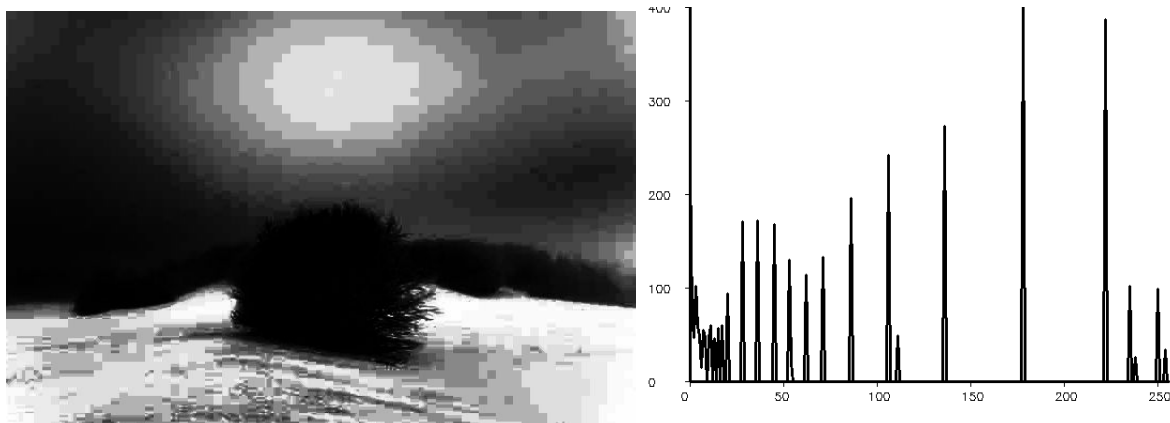


Figura 15: Primera imagen ecualizada *rayleigh* con $\sigma = 2$ y su histograma.

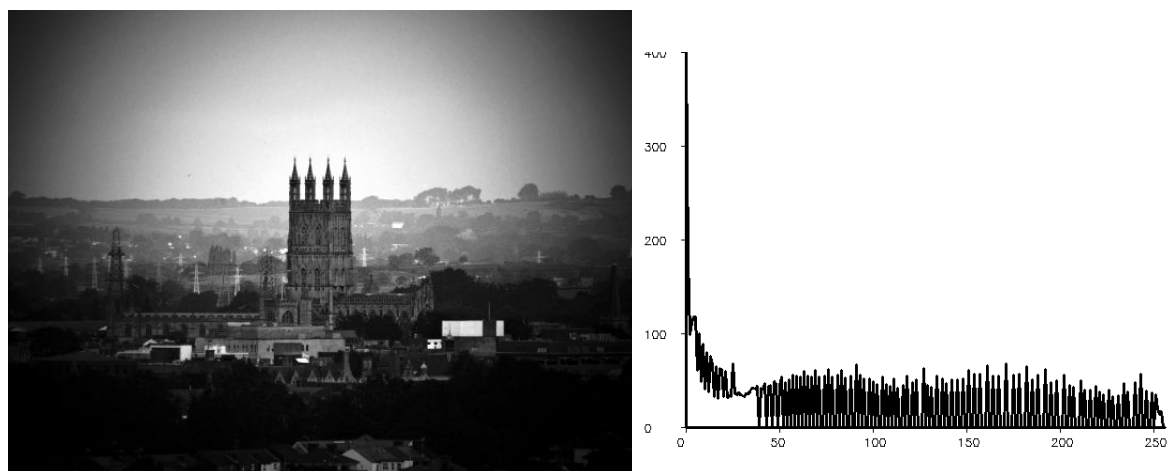


Figura 16: Segunda imagen ecualizada *rayleigh* con $\sigma = 2$ y su histograma.

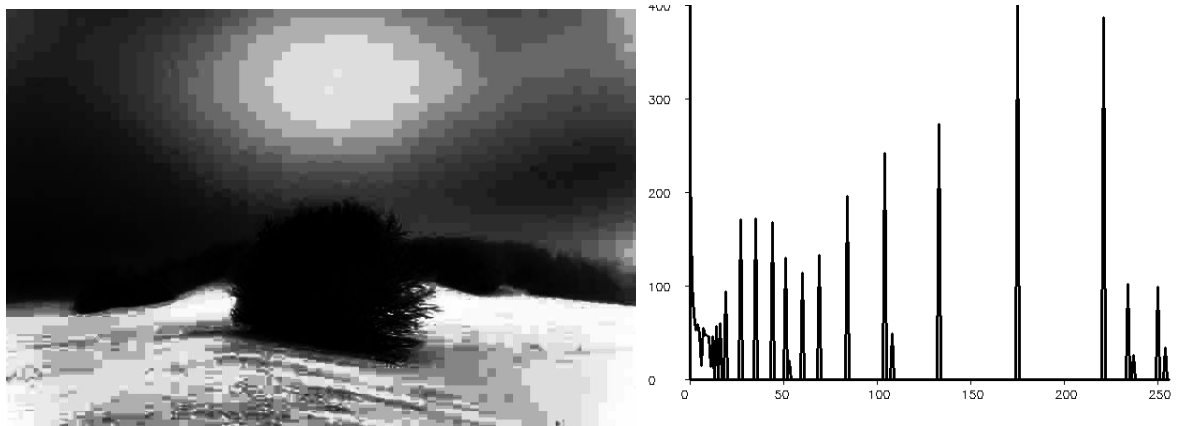


Figura 17: Primera imagen ecualizada *rayleigh* con $\sigma = 3,5$ y su histograma.

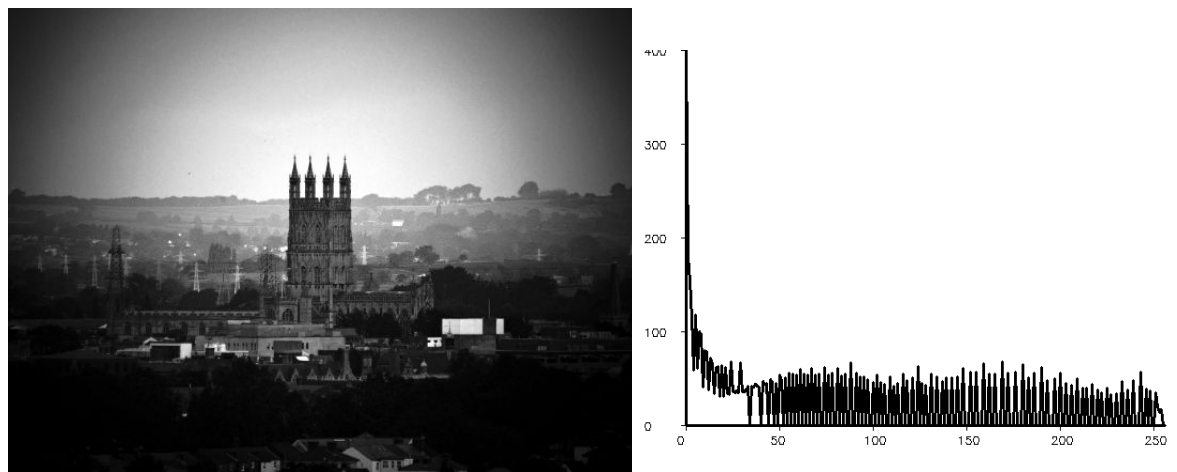


Figura 18: Segunda imagen ecualizada *rayleigh* con $\sigma = 3,5$ y su histograma.

- Ecualización *gamma* con $\gamma = 1, 2, 3$.

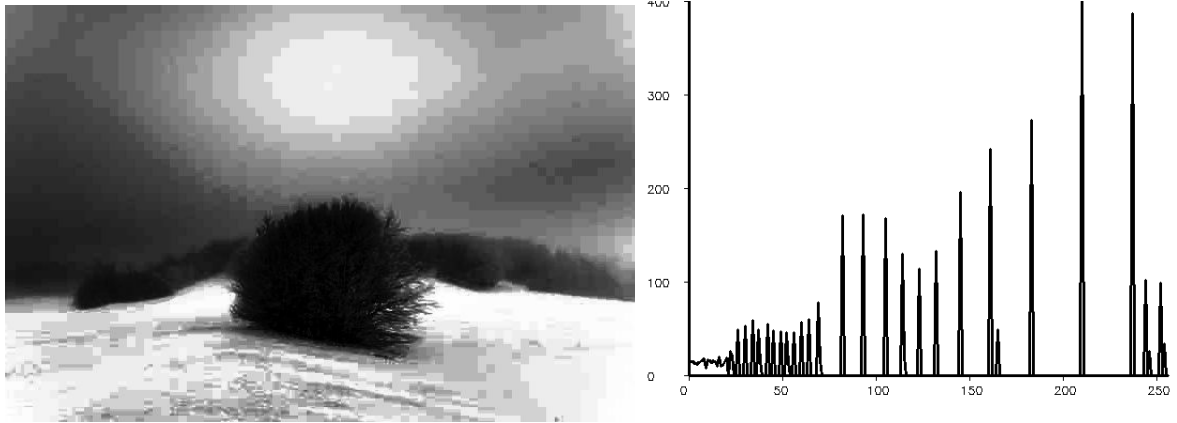


Figura 19: Primera imagen ecualizada con $\gamma = 1$ y su histograma.

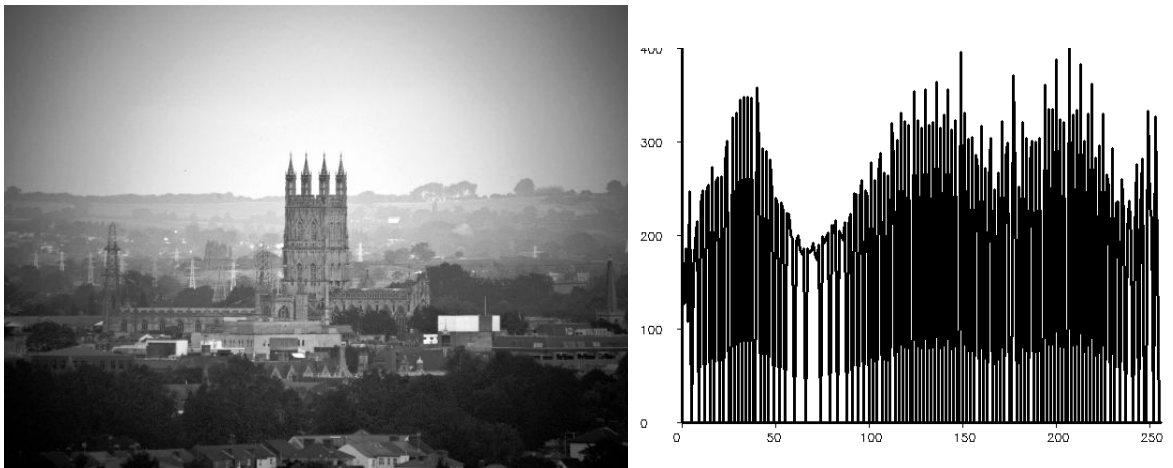


Figura 20: Segunda imagen ecualizada con $\gamma = 1$ y su histograma.

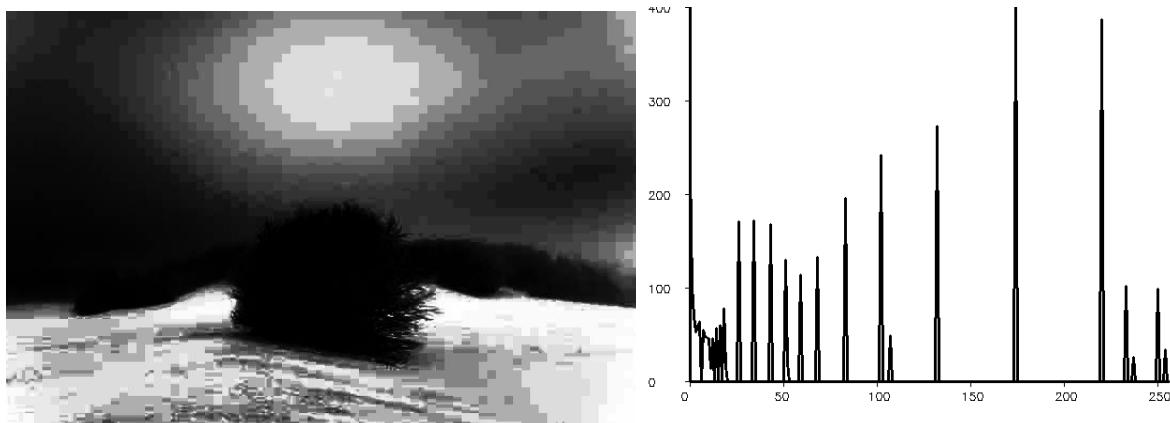


Figura 21: Primera imagen ecualizada con $\gamma = 2$ y su histograma.

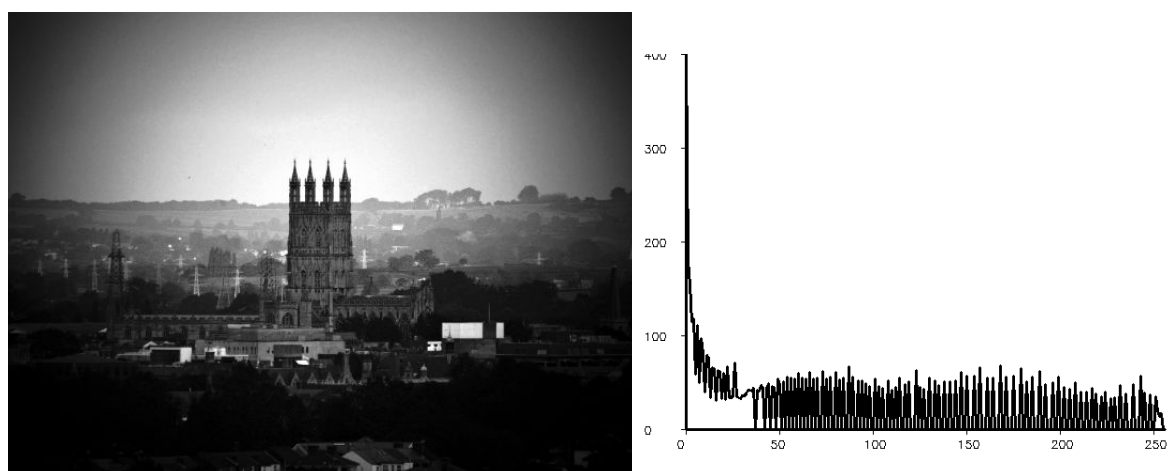


Figura 22: Segunda imagen ecualizada con $\gamma = 2$ y su histograma.

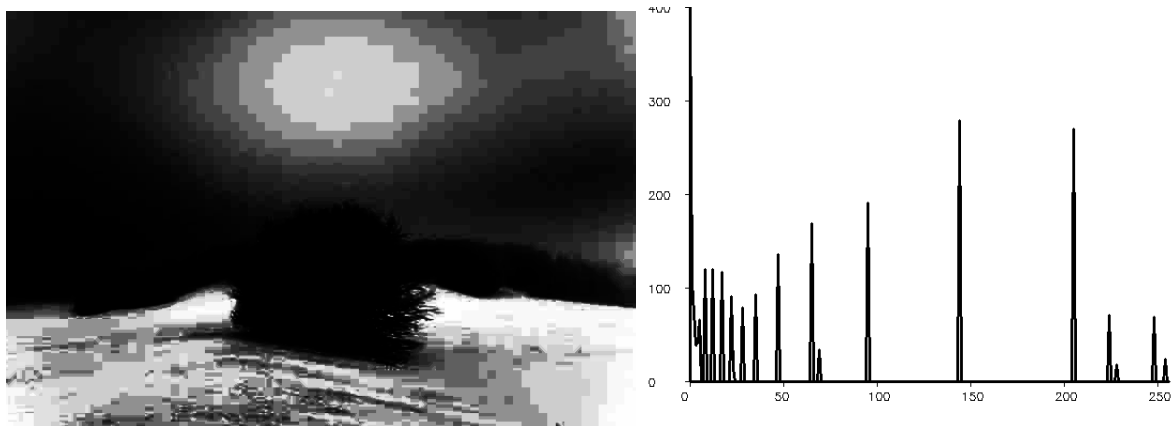


Figura 23: Primera imagen ecualizada con $\gamma = 3$ y su histograma.

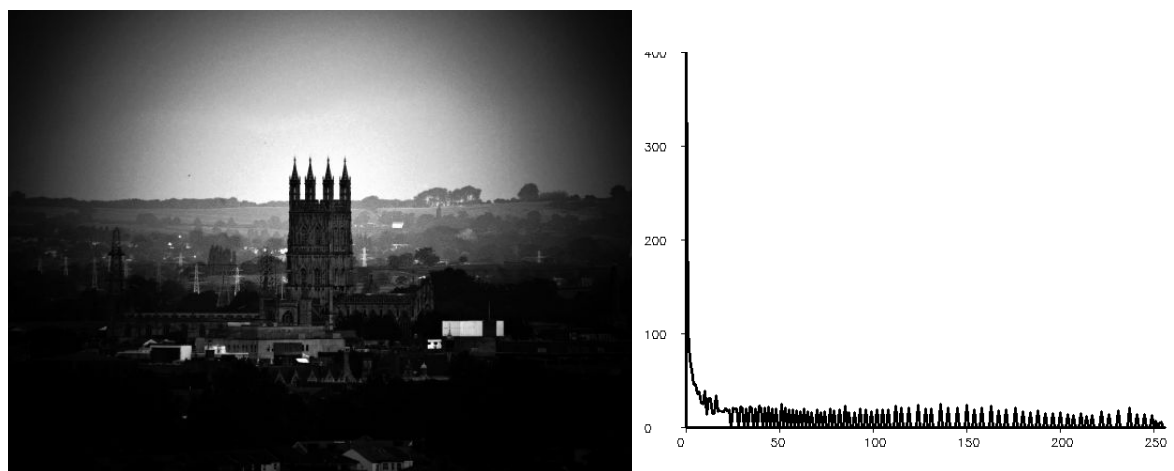


Figura 24: Segunda imagen ecualizada con $\gamma = 3$ y su histograma.

- Ecualización logarítmica normalizada con $\alpha = 1, 2, 3$.

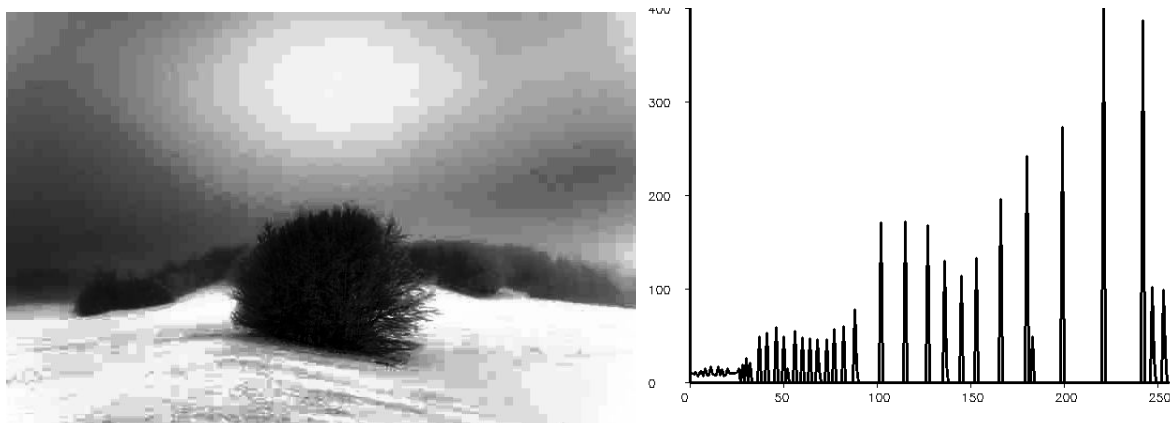


Figura 25: Primera imagen ecualizada logarítmicamente con $\alpha = 1$ y su histograma.

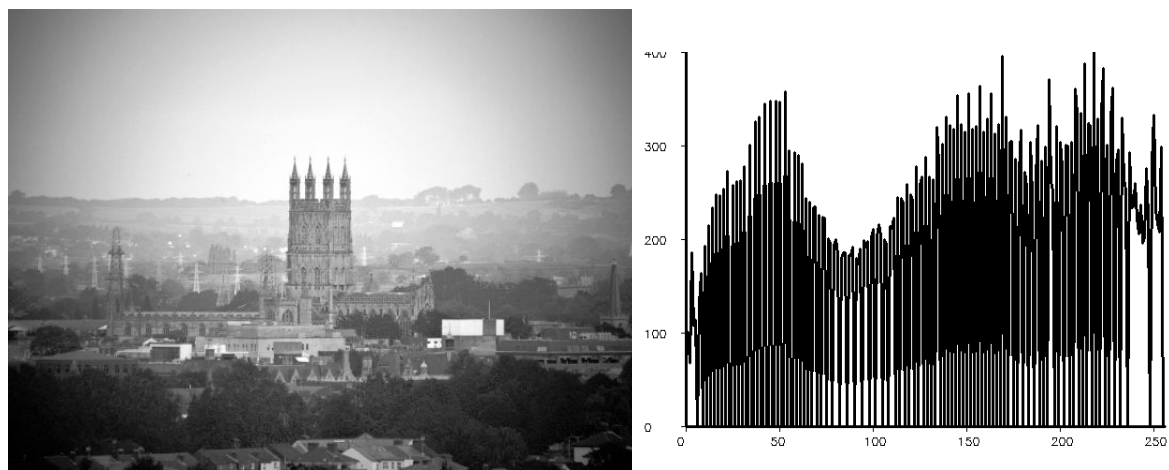


Figura 26: Segunda imagen ecualizada logarítmicamente con $\alpha = 1$ y su histograma.

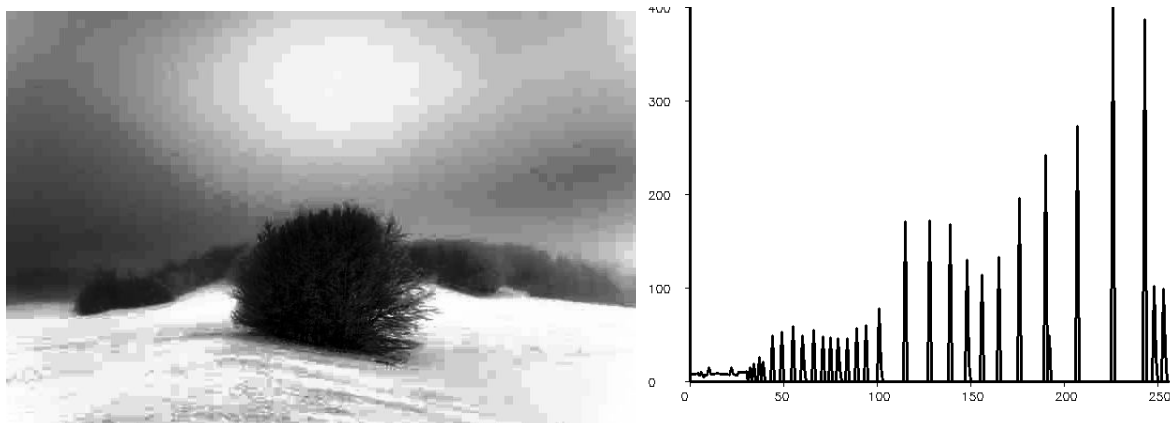


Figura 27: Primera imagen ecualizada logarítmicamente con $\alpha = 2$ y su histograma.

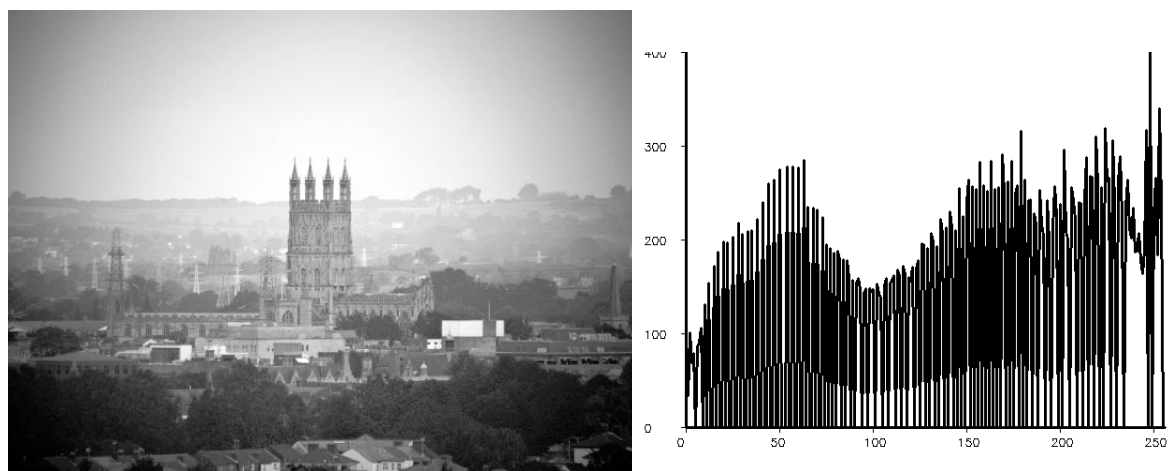


Figura 28: Segunda imagen ecualizada logarítmicamente con $\alpha = 2$ y su histograma.

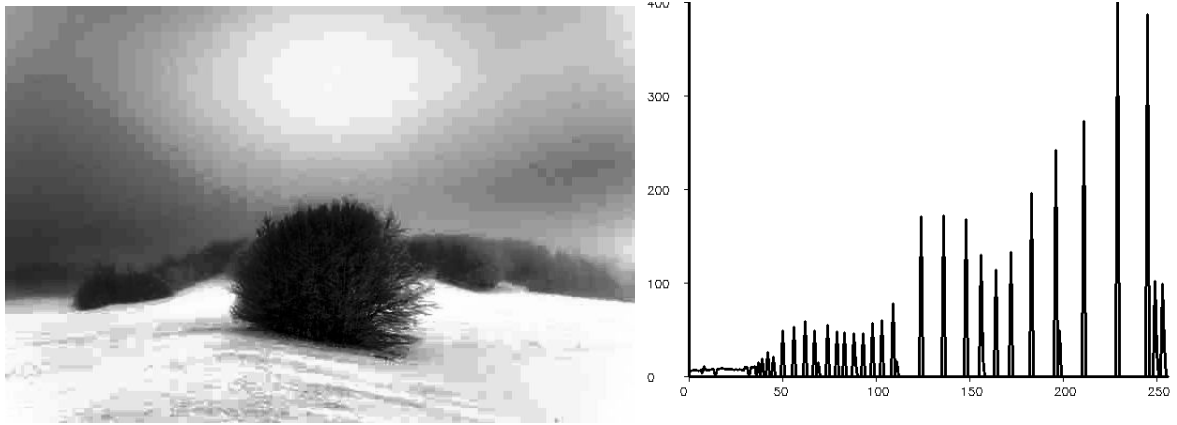


Figura 29: Primera imagen ecualizada logarítmicamente con $\alpha = 3$ y su histograma.

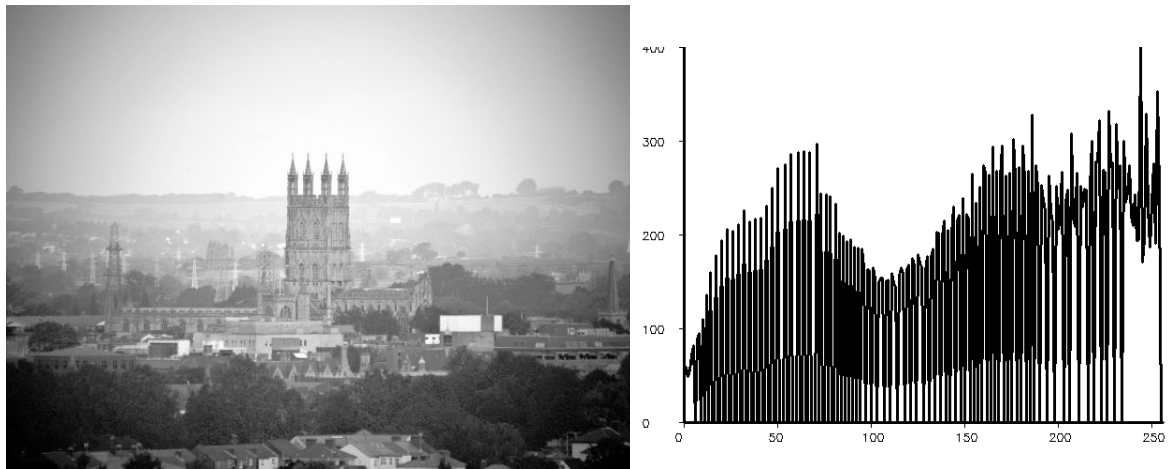


Figura 30: Segunda imagen ecualizada logarítmicamente con $\alpha = 3$ y su histograma.

Distribución	Transformación	Efecto
Uniforme	$s = (L - 1) P_r(r_k)$	Genera un histograma lo más plano o uniforme posible y supone una mejora general del contraste. Se usa en imágenes con bajo contraste global.
Exponencial	$s = (L - 1) \frac{1 - e^{-\alpha P_r(r_k)}}{1 - e^{-\alpha}}$	Crece muy rápido al principio, es decir, en sombras o intensidades r_k bajas y aumenta su contraste. En intensidades altas o luces se comprime o aplana reduciendo el contraste. El parámetro α controla la curvatura de la función y por tanto, la intensidad de realzado de las sombras. Con α cercano a 0 se comporta similar a una transformación lineal, con α cercano a 1 las sombras se realzan suavemente y con $\alpha > 1$ se realzan fuertemente.
Rayleigh	$s = (L - 1) \frac{1 - \exp\left(-\frac{P_r(r_k)^2}{2\sigma^2}\right)}{1 - \exp\left(-\frac{1}{2\sigma^2}\right)}$	Similar a la exponencial, en lugar de realzar las intensidades bajas y comprimir las altas, da mayor peso a las intensidades r_k medias. El parámetro σ controla la anchura de la distribución: valores grandes realzan el contraste entre tonos medios.
Transformación gamma, hipercúbica con $\gamma = 3$	$s = (L - 1) P_r(r_k)^\gamma$	Si $\gamma < 1$ realza las sombras o intensidades bajas r_k aumentando su contraste; si $\gamma > 1$ realza las luces o intensidades altas r_k . Con $\gamma = 1$ la transformación es lineal.
Logarítmica normalizada o logarítmica hiperbólica	$s(r) = (L - 1) \frac{\log(1 + \alpha P_r(r_k))}{\log(1 + \alpha)}$	Similar a la exponencial, pero más suave. El parámetro $\alpha > 0$ controla la curvatura, a mayor valor mayor brusquedad en el aumento de contraste de las intensidades bajas r_k .

Cuadro 10: Funciones de ecualización más comunes.

Registro automático entre imágenes

Es una técnica cuyo objetivo es **alinear dos imágenes de la misma escena** tomadas en condiciones diferentes de perspectiva, rotación, escala o iluminación. El proceso consiste en encontrar correspondencias entre ambas imágenes y calcular una transformación geométrica que permita superponerlas de manera coherente.

En este caso se ha utilizado el método basado en **detección de características locales** con el detector *ORB (Oriented FAST and Rotated BRIEF)*, ya implementado en la librería *OpenCV*. Los pasos principales son:

- **Detección de puntos clave:** se localizan esquinas y estructuras distintivas en ambas imágenes. Se toma un píxel candidato y se compara con los píxeles de un círculo alrededor de él. Si un conjunto de píxeles consecutivos en ese círculo son todos más brillantes o más oscuros que el píxel central por un cierto umbral, se considera un punto clave. Una técnica popular para esta detección es el detector de Harris, implementado en el tema 4. En esta implementación se utiliza el umbral por defecto, es decir, 20.
- **Extracción de descriptores:** se codifica la vecindad de cada punto clave en un vector binario, generado a partir de comparaciones entre pares de píxeles vecinos. Para cada par, se asigna un bit con valor 1 si la intensidad del primer píxel es menor que la del segundo y 0 en caso contrario.
- **Emparejamiento de descriptores:** se buscan correspondencias entre los descriptores de ambas imágenes. El objetivo es que dos puntos similares en distintas imágenes tengan descriptores parecidos, aunque haya rotación, escala o cambios de iluminación.
- **Cálculo de homografía:** la homografía es una transformación proyectiva que relaciona dos planos y permite mapear puntos de una imagen a otra cuando ambas representan el mismo plano del mundo real, pero vistas desde diferentes perspectivas. Se estima la matriz de transformación homográfica 3×3 que relaciona los puntos de la imagen móvil con los de la referencia:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} \sim H \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

donde (x, y) son las coordenadas originales y (x', y') las coordenadas transformadas. Al emplear la librería *OpenCV* se ha utilizado *RANSAC* para eliminar emparejamientos erróneos.

- **Transformación:** se aplica la homografía a la imagen móvil para alinearla con la referencia.

Algoritmo, implementación y codificación

1. Se cargan las dos imágenes: la referencia y la móvil.
2. Se detectan puntos clave y descriptores con *ORB* en ambas imágenes.
3. Se emparejan los descriptores con *BFMatcher* y se ordenan por distancia.
4. Se conserva un porcentaje de las mejores coincidencias (por defecto, 0,15).

5. Se extraen las coordenadas de los puntos emparejados y se calcula la homografía con RANSAC.
6. Se aplica la transformación a la imagen móvil mediante `warpPerspective`.
7. Finalmente, se guarda la imagen registrada.

La implementación se ha realizado en **C++** utilizando la librería *OpenCV*, aprovechando las funciones `ORB::create()`, `BFMatcher`, `findHomography()` y `warpPerspective()`.

Resultados

Se ha realizado el registro entre dos imágenes tomadas con un dispositivo móvil. Ambas imágenes, junto con el resultado, se presentan en la figura 31. Se observa que la segunda imagen se ha alineado con la referencia, de forma que las estructuras comunes, es decir, las que aparecen en ambas (en este caso, la caja de la maqueta del avión) quedan superpuestas, y el trozo de pared verde solo visible en la segunda imagen, se alinea y se añade al resultado.



Figura 31: Resultado del registro automático entre imágenes.

Tema 4: extracción de bordes, regiones y puntos de interés.

Extracción de bordes mediante operaciones en la primera derivada: Sobel

La detección de bordes mediante el **operador de Sobel** se basa en el cálculo de la primera derivada de la imagen. Un borde se caracteriza por un **cambio brusco de intensidad, lo que se traduce en un máximo o mínimo en la primera derivada**. El operador Sobel aproxima estas derivadas en las direcciones x e y mediante filtros discretos que combinan una máscara de derivada con un ligero suavizado, lo que lo hace más robusto frente al ruido que una simple diferencia finita.

Los filtros Sobel clásicos son:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}.$$

Aplicando estas máscaras se obtienen las derivadas aproximadas en cada dirección:

$$I_x(x, y) = f(x, y) * G_x, \quad I_y(x, y) = f(x, y) * G_y,$$

donde $*$ denota la convolución. A partir de ellas se calcula la magnitud del gradiente:

$$M(x, y) = \sqrt{I_x(x, y)^2 + I_y(x, y)^2},$$

que indica la fuerza del borde en cada píxel. Opcionalmente se puede aplicar un umbral para obtener una imagen binaria de bordes.

Algoritmo, implementación y codificación

1. Se carga la imagen original $f(x, y)$ en escala de grises.
2. Se aplica la **convolución con las máscaras Sobel** G_x y G_y para obtener las derivadas aproximadas I_x e I_y .
3. Se calcula la **magnitud** del gradiente.
4. Se normaliza la magnitud al rango $[0, 255]$ para visualizarla como imagen.
5. Opcionalmente, se aplica un umbral T para obtener una imagen binaria de bordes:

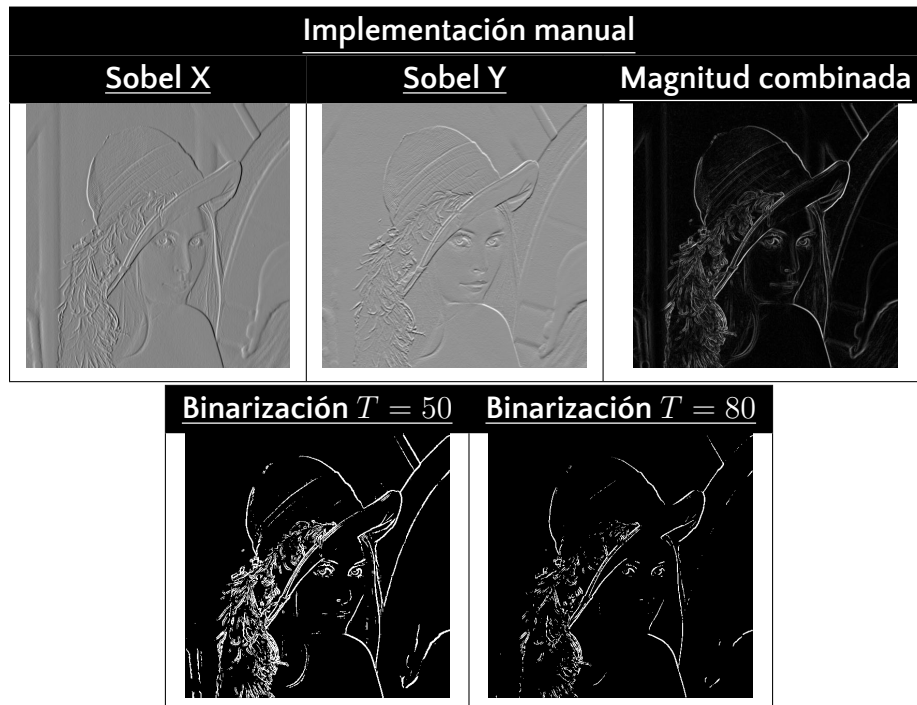
$$M(x, y) \geq T \Rightarrow \text{borde}.$$

La codificación e implementación del método se ha realizado en el lenguaje **C++** con la ayuda de la librería *OpenCV*. Se han desarrollado dos versiones:

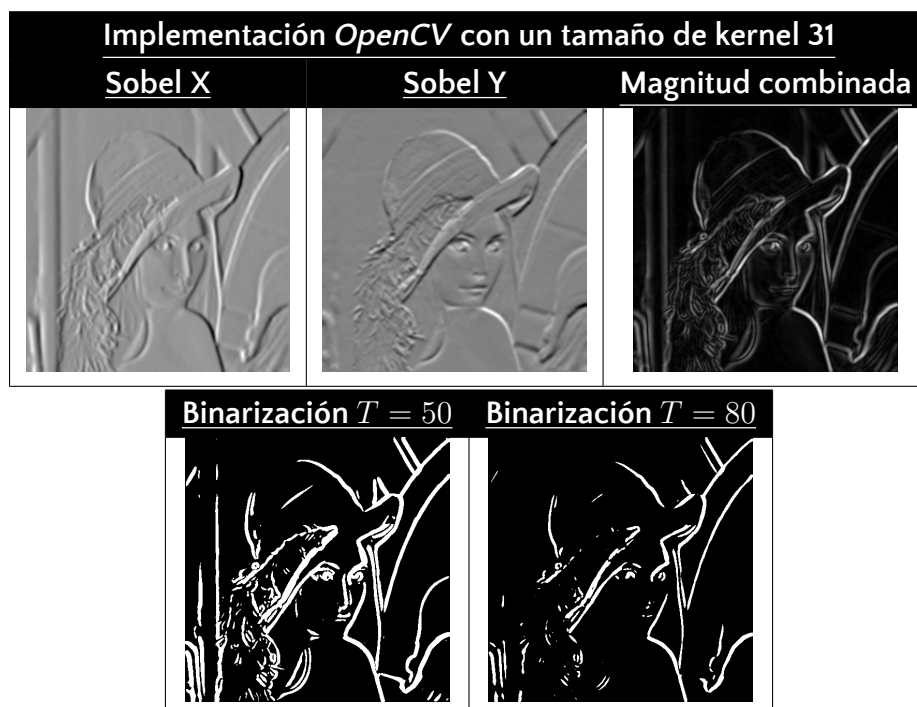
- Una implementación manual mediante `filter2D` y los kernels Sobel definidos explícitamente.
- Una implementación directa mediante la función `cv::Sobel`, que permite variar el tamaño del kernel y el orden de la derivada.

Resultados

Se han extraído los bordes de la imagen de Lena aplicando el operador Sobel. Los resultados se muestran en las tablas 11 y 12, donde se comparan las imágenes obtenidas con las dos implementaciones. Se observa el efecto del tamaño del kernel, ya que al ser muy grande el efecto de suavizado se realza, difuminando los bordes y consecuentemente, los bordes son más gruesos.



Cuadro 11: Resultados de la implementación manual del operador Sobel.



Cuadro 12: Resultados de la implementación *OpenCV* del operador Sobel.

Extracción de puntos de interés mediante Harris

La detección de esquinas o puntos de interés con el **detector de Harris** se basa en analizar cómo **cambia la intensidad** de una imagen al desplazar una pequeña ventana por distintas direcciones. Una esquina se identifica cuando la intensidad varía de forma notable en dos direcciones independientes, lo que se refleja en **valores elevados de los autovalores de la matriz de momentos locales**. Esta matriz se calcula **a partir de las derivadas parciales** de la imagen, promediadas dentro de una ventana con ponderación gaussiana, lo que hace al método más estable frente al ruido y ligeras transformaciones geométricas.

Sea $I(x, y)$ la imagen en escala de grises, definimos las derivadas parciales aproximadas I_x e I_y . La matriz de momentos locales en una vecindad alrededor de (x, y) es

$$M(x, y) = \begin{bmatrix} \sum w(u, v) I_x^2(u, v) & \sum w(u, v) I_x(u, v) I_y(u, v) \\ \sum w(u, v) I_x(u, v) I_y(u, v) & \sum w(u, v) I_y^2(u, v) \end{bmatrix},$$

donde la suma se realiza sobre la ventana y $w(u, v)$ es una ventana de ponderación, típicamente una gaussiana. A partir de M se calcula la **respuesta de Harris**

$$R(x, y) = \det(M(x, y)) - k (\text{traza}(M(x, y)))^2,$$

donde k es un parámetro empírico normalmente en el rango $[0,04; 0,06]$ que controla la sensibilidad del algoritmo y cuyos valores bajos hacen al detector más propenso a identificar esquinas agudas, pero también puede incrementar la detección de falsos positivos (bordes que parecen esquinas). **Valores de R grandes y positivos indican esquinas**; valores negativos indican bordes dominantes y valores cerca de cero regiones planas.

Algoritmo, implementación y codificación

1. Se carga la imagen original $I(x, y)$ en escala de grises.
2. Se calculan las **derivadas I_x e I_y mediante convoluciones** (por ejemplo, con Sobel) o usando la función interna de OpenCV.
3. Se construyen los elementos de la **matriz de momentos locales** I_x^2 , $I_x I_y$ e I_y^2 y se aplica una **ventana gaussiana** para obtener las sumas locales.
4. Se calcula la **respuesta de Harris** $R(x, y) = \det(M) - k \text{traza}(M)^2$.
5. Se normaliza o escala R para visualización y se aplica un umbral T_R para seleccionar candidatos a esquinas.

Se ha realizado la implementación en **C++** con *OpenCV* utilizando la función `cv::cornerHarris(src, dst, blockSize, ksize, k)`, donde `blockSize` es el tamaño de la vecindad para la suma, `ksize` el tamaño del derivador (Sobel) y `k` el parámetro de Harris.

Resultados

Se ha aplicado el detector de Harris a la imagen de Lena con diferentes parámetros de `blockSize` y `ksize`, ya que el tamaño del operador de Sobel se ha estudiado en el apartado anterior, y a una imagen propia. Se muestra la visualización de la respuesta de Harris y la imagen original con los puntos de interés superpuestos. Los resultados se presentan en la tabla 13, donde se observa que aumentar `blockSize` o la ponderación de la gaussiana produce detecciones más estables pero

reduce la sensibilidad a esquinas finas y modificar k , en las imágenes probadas, no tiene un gran impacto.

blockSize = 2, ksize = 3, k = 0,04		blockSize = 24, ksize = 3, k = 0,04	
Respuesta Harris	Puntos superpuestos	Respuesta Harris	Puntos superpuestos
blockSize = 2, ksize = 3, k = 0,06		blockSize = 24, ksize = 3, k = 0,06	
Respuesta Harris	Puntos superpuestos	Respuesta Harris	Puntos superpuestos
blockSize = 18, ksize = 3, k = 0,06			
Respuesta Harris		Puntos superpuestos	

Cuadro 13: Resultados del detector de Harris.

Extracción de bordes mediante operaciones en la segunda derivada: aproximación de Kim y Aggarwal

La detección de bordes mediante la **laplaciana** se basa en el análisis de la **segunda derivada** de la imagen. Un borde se caracteriza por un cambio brusco de intensidad, lo que se traduce en un máximo o mínimo en la primera derivada. En la segunda derivada, este máximo o mínimo se convierte en un **cambio de signo**, lo que se conoce como **cruce por cero**, y coincide con el centro del borde permitiendo localizarlo con mayor precisión para resaltar las regiones donde la intensidad cambia de manera abrupta.

El operador laplaciano combina las segundas derivadas en las direcciones x e y de la imagen:

$$\nabla^2 f(x, y) = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}.$$

Dicha laplaciana se calcula aplicando los filtros discretos de aproximación de segunda derivada:

Nombre	Filtro	Efecto
Laplaciano 4-conexo	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	Solo considera los píxeles verticales y horizontales alrededor del centro. Detecta bordes más finos y es menos sensible al ruido lateral.
Laplaciano 8-conexo	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	Incluye las diagonales. Detecta bordes más gruesos y es más sensible al ruido. Útil para capturar bordes en todas las direcciones.
Laplaciano ponderado	$\begin{bmatrix} 1 & -2 & 1 \\ -2 & 4 & -2 \\ 1 & -2 & 1 \end{bmatrix}$	Da más peso a los píxeles cercanos al centro. Mejora la detección de bordes en imágenes con gradientes suaves. Es una aproximación más precisa de la segunda derivada.

Cuadro 14: Filtros discretos de aproximación de la Laplaciana y su efecto en la detección de bordes.

Una consecuencia indeseada de aplicar derivadas es realzar el ruido de la imagen, es decir, las variaciones rápidas de la señal. En este caso, como se trata de la segunda derivada, resulta necesario aplicar un filtro gaussiano para suavizar o contrarrestar esas frecuencias altas. Por tanto, normalmente, la laplaciana se suele aplicar tras un suavizado gaussiano, lo que se conoce como **Laplaciano de Gauss (LoG)**.

Además, con el objetivo de aumentar la robustez de la detección de los bordes y disminuir el riesgo de detectar falsos bordes, la **aproximación de Kim y Aggarwal** propone un criterio adicional para decidir si un cruce por cero corresponde realmente a un borde: se exige que la diferencia de magnitudes entre los valores de signo opuesto supere un umbral relativo para evitar falsos positivos en regiones planas debidas al ruido.

Algoritmo, implementación y codificación

1. Se suaviza la imagen original $f(x, y)$ con un **filtro gaussiano** de desviación estándar σ . Se reutiliza la implementación hecha en el tema 3.

2. Se calcula la **Laplaciana** de la imagen suavizada aplicando el filtro de convolución $h(x, y)$:

$$L(x, y) = \nabla^2 g(x, y) = g(x, y) * h(x, y)$$

3. Para cada píxel, se comprueba si existe un **cambio de signo** en $L(x, y)$ respecto a alguno de sus vecinos k -conexos. Si lo hay, se considera un posible cruce por cero.
4. Según Kim y Aggarwal, se calcula la **diferencia de magnitudes** entre los valores de signo opuesto. Solo se marca como borde si esta diferencia **supera un umbral relativo T** :

$$|L(x, y)| + |L(x', y')| \geq T,$$

donde (x', y') es un vecino con signo opuesto.

5. El resultado es una imagen binaria donde los píxeles de borde se representan en blanco (valor 255) y el resto en negro (valor 0).

La codificación e implementación del método se ha realizado en el lenguaje **C++** con la ayuda de la librería *OpenCV*. Se ha reutilizado la implementación del filtro gaussiano del tema 3 .

Resultados

Se han extraído los bordes de la imagen de Lena aplicando diferentes valores a los parámetros. Los resultados se recogen en la tabla 15. Se observa que el filtro ponderado detecta mejor los bordes suaves, pero amplifica el ruido y los bordes fuertes y que el filtro 8-conexo detecta más detalles en los bordes que el filtro 4-conexo.

$\sigma = 0,5, T = 5, 4\text{-conexo}$	$\sigma = 0,5, T = 5, \text{ponderado}$	$\sigma = 0,5, T = 5, 8\text{-conexo}$
		
$\sigma = 0,5, T = 10, 4\text{-conexo}$	$\sigma = 0,5, T = 10, \text{ponderado}$	$\sigma = 0,5, T = 10, 8\text{-conexo}$
		

Cuadro 15: Resultados de la aproximación de Kim y Aggarwal para la extracción de bordes para distintos valores de los parámetros.

Tema 5: descripción de bordes y regiones.

Transformada de Hough

La **transformada de Hough** se basa en la representación de las rectas y otras curvas en un espacio de parámetros para detectar formas geométricas. El objetivo es **localizar líneas, circunferencias o elipses** en una imagen, incluso cuando los bordes están incompletos o afectados por ruido.

Una recta en el espacio de la imagen puede expresarse como $y = mx + b$, donde m es la pendiente y b la ordenada en el origen. Sin embargo, esta representación presenta un problema cuando la recta es vertical, es decir, $x = c$ constante, ya que la variación en x es nula y la pendiente tiende a infinito:

$$m = \frac{\Delta y}{\Delta x} = \frac{\Delta y}{0} = \infty.$$

Para evitar este inconveniente se utiliza la **forma polar** de la recta, $\rho = x \cos \theta + y \sin \theta$, donde ρ es la distancia perpendicular de la recta al origen en el rango $\rho \in [-D, D]$, donde $D = \sqrt{(\text{ancho})^2 + (\text{alto})^2}$, esto es, la diagonal de la imagen; y θ es el ángulo que forma la normal de la recta con el eje x . Esta representación es válida para cualquier orientación de la recta con $\theta \in [0, \pi)$, incluidas las verticales y horizontales, y no depende de la pendiente.

Cada punto o píxel de borde (x, y) detectado en la imagen se transforma en una **curva sinusoidal** distinta en el espacio de parámetros (ρ, θ) , es decir, fijamos las componentes x e y en la ecuación polar $\rho = x \cos \theta + y \sin \theta$, y variando θ en el rango mencionado se obtiene la curva sinusoidal. Los píxeles que están alineados en una misma forma geométrica formarán sinusoides que se cruzan en un mismo punto (ρ^*, θ^*) . Ese punto común es la representación de la recta original en el espacio de parámetros.

Para materializar este proceso se utiliza una **matriz acumuladora**, donde cada celda corresponde a un par de parámetros discretos (ρ, θ) . Cada vez que un píxel de borde (x, y) es consistente con una recta descrita por esos parámetros, se incrementa en uno el valor de la celda correspondiente. A este incremento se le denomina **voto del píxel**.

De este modo, los parámetros (ρ, θ) que reciben más votos en el acumulador corresponden a las rectas que mejor explican la alineación de los píxeles en la imagen. Los **picos en el acumulador** son, por tanto, las rectas detectadas.

Finalmente, uniendo los píxeles de borde asociados a esos parámetros se reconstruye la forma geométrica que dibujan en la imagen original.

En el caso de circunferencias o elipses, el principio es el mismo, pero con más parámetros: centro, radio... Por ejemplo, para identificar circunferencias se emplea la ecuación que las define: $(x - a)^2 + (y - b)^2 = r^2$, donde (a, b) es el centro y r el radio. El espacio de parámetros deja de ser bidimensional (ρ, θ) y pasa a ser tridimensional (a, b, r) .

Cada píxel de borde (x, y) puede pertenecer a múltiples circunferencias. Para cada valor de r considerado, se calculan los posibles centros (a, b) que satisfacen la ecuación, y se incrementa

el valor del acumulador tridimensional en esas posiciones. Los picos en el acumulador (a^*, b^*, r^*) corresponden a circunferencias presentes en la imagen. Así, varios puntos de borde que pertenecen a la misma circunferencia generan superficies en el espacio de parámetros que se cruzan en un mismo punto.

La transformada de Hough es **robusta al ruido y a bordes rotos o incompletos**, puesto que es suficiente con que un número determinado de píxeles de borde voten de manera coherente en el espacio de parámetros para que aparezca un pico en el acumulador que identifique la figura global. Además, es generalizable a distintas curvas paramétricas. Consecuentemente, es especialmente útil para la detección de carriles en carreteras o el análisis de piezas industriales.

No obstante, también presenta algunas **limitaciones**: el coste computacional puede ser elevado debido al tamaño del espacio de parámetros y la resolución depende de la discretización del espacio de parámetros.

Cabe destacar los diferentes parámetros con los que se ha implementado este método:

Parámetros de la Transformada de Hough para rectas

- θ_{bins} : número de divisiones del ángulo $\theta \in [0, \pi)$. Una discretización más fina aumenta la precisión angular, pero también el coste computacional.
- $\Delta\rho$: paso de discretización de la distancia $\rho \in [-D, D]$. Controla la resolución espacial en el acumulador.
- T_{line} : umbral mínimo de votos en el acumulador para aceptar una recta. Rectas con menos de T_{line} votos se descartan como ruido.
- NMS_{line} : tamaño del vecindario usado en la supresión de no-máximos para garantizar que solo se seleccionen los picos más significativos. Cuando varios píxeles de borde votan por una recta, en el acumulador aparece un pico en la celda correspondiente. Debido a la discretización, ese pico no suele concentrarse en una sola celda, sino que se esparce en celdas vecinas. Este parámetro evita identificar rectas idénticas. Cuando este parámetro vale 3, significa que compara cada pico con un bloque de 7x7 a su alrededor, puesto que se toma ± 3 en cada dirección.
- $N_{\text{max-lines}}$: número máximo de rectas a mostrar o reconstruir en la imagen.

Parámetros de la Transformada de Hough para circunferencias

- $r_{\text{mín}}, r_{\text{máx}}, \Delta r$: rango de radios y paso de discretización. Determinan qué tamaños de circunferencia se buscan en la imagen.
- $\theta_{\text{circle-bins}}$: número de divisiones angulares utilizadas para votar posibles centros (a, b) de la circunferencia. Normalmente se toma $\theta \in [0, 2\pi)$.
- T_{circle} : umbral mínimo de votos en el acumulador tridimensional (a, b, r) para aceptar una circunferencia.
- NMS_{circle} : tamaño del vecindario para la supresión de no-máximos en el acumulador 3D.
- $N_{\text{max-circles}}$: número máximo de circunferencias a mostrar o reconstruir en la imagen.

Algoritmo, implementación y codificación

1. Se **detectan los bordes** de la imagen. Lo más común es utilizar Canny o Sobel, pero en esta implementación se reutiliza la detección de bordes mediante la aproximación de Kim y Aggarwal del tema 4.
2. Cada punto de borde (x,y) se transforma en una **curva paramétrica y espacio** correspondientes.
3. Se **discretiza** el espacio correspondiente y se genera el **acumulador**. Cada píxel aumenta el valor del acumulador en las celdas correspondientes.
4. Se buscan los **máximos** en el acumulador, que representan figuras geométricas presentes en la imagen.
5. Para el conjunto de valores seleccionado del acumulador, se construye la figura geométrica identificada y se superpone a la imagen original para resaltarla.

La codificación e implementación del método se ha realizado en el lenguaje **C++** con la ayuda de la librería *OpenCV*. No obstante, se ha evitado el uso de las funciones *HoughLines* y *HoughCircles* con el objetivo de implementar manualmente el método.

Resultados

Se han extraído las rectas de la imagen original 32 y las circunferencias de la imagen original 33. En ambos casos se han utilizado diferentes parámetros y se presentan los resultados de la laplaciana y de la transformada de Hough en las tablas 16 y 17.



Figura 32: Primera imagen original de la que se han extraído las rectas.

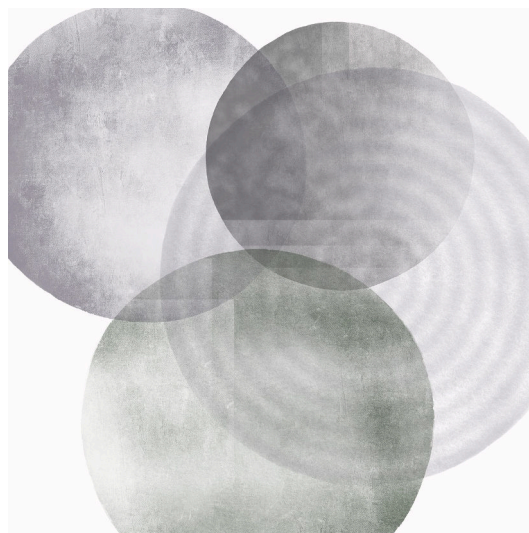
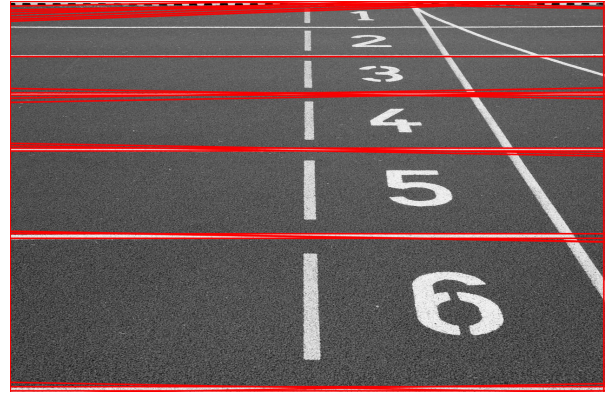
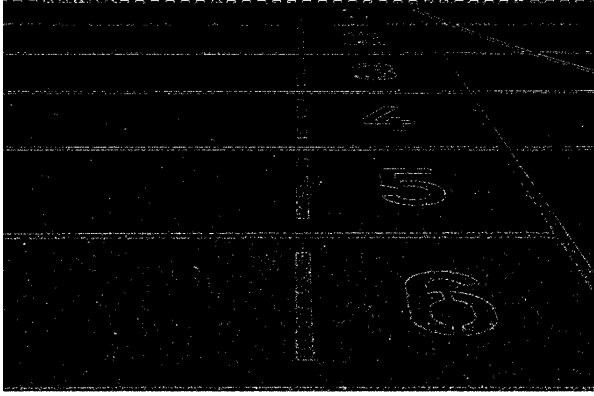


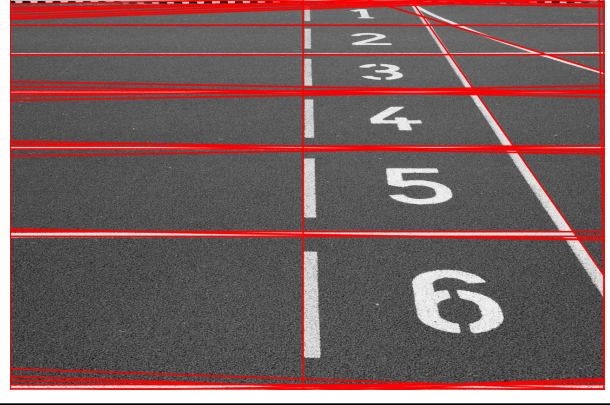
Figura 33: Segunda imagen original de la que se han extraído las circunferencias.

$\sigma = 0,5$, $T = 10$, 4-conexo

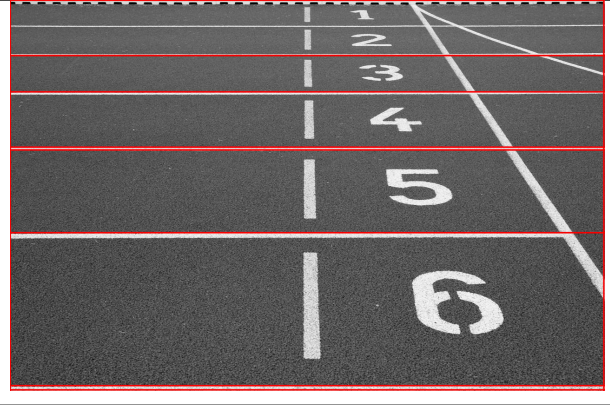
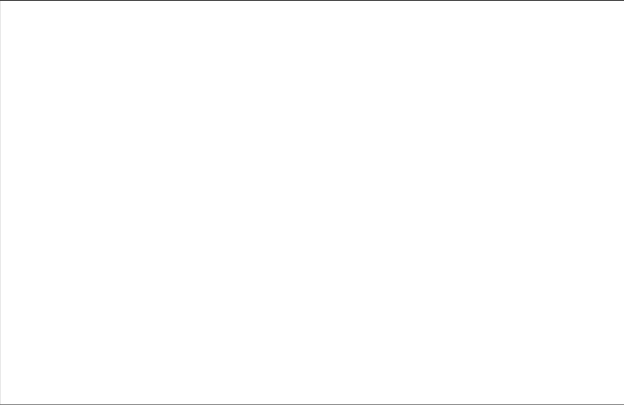
$\theta_{\text{bins}} = 180$; $\Delta\rho = 1$; $T_{\text{line}} = 80$; $N_{\text{max-lines}} = 30$



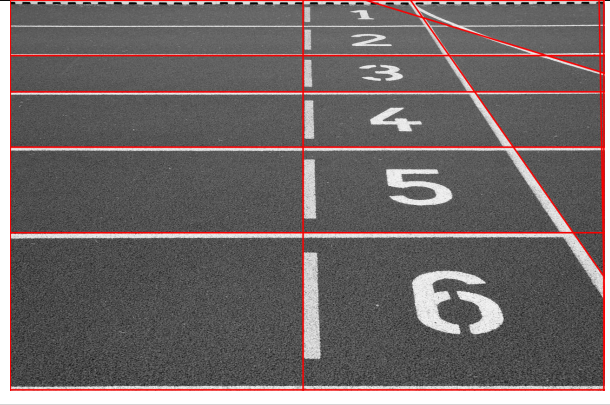
$N_{\text{max-lines}} = 200$

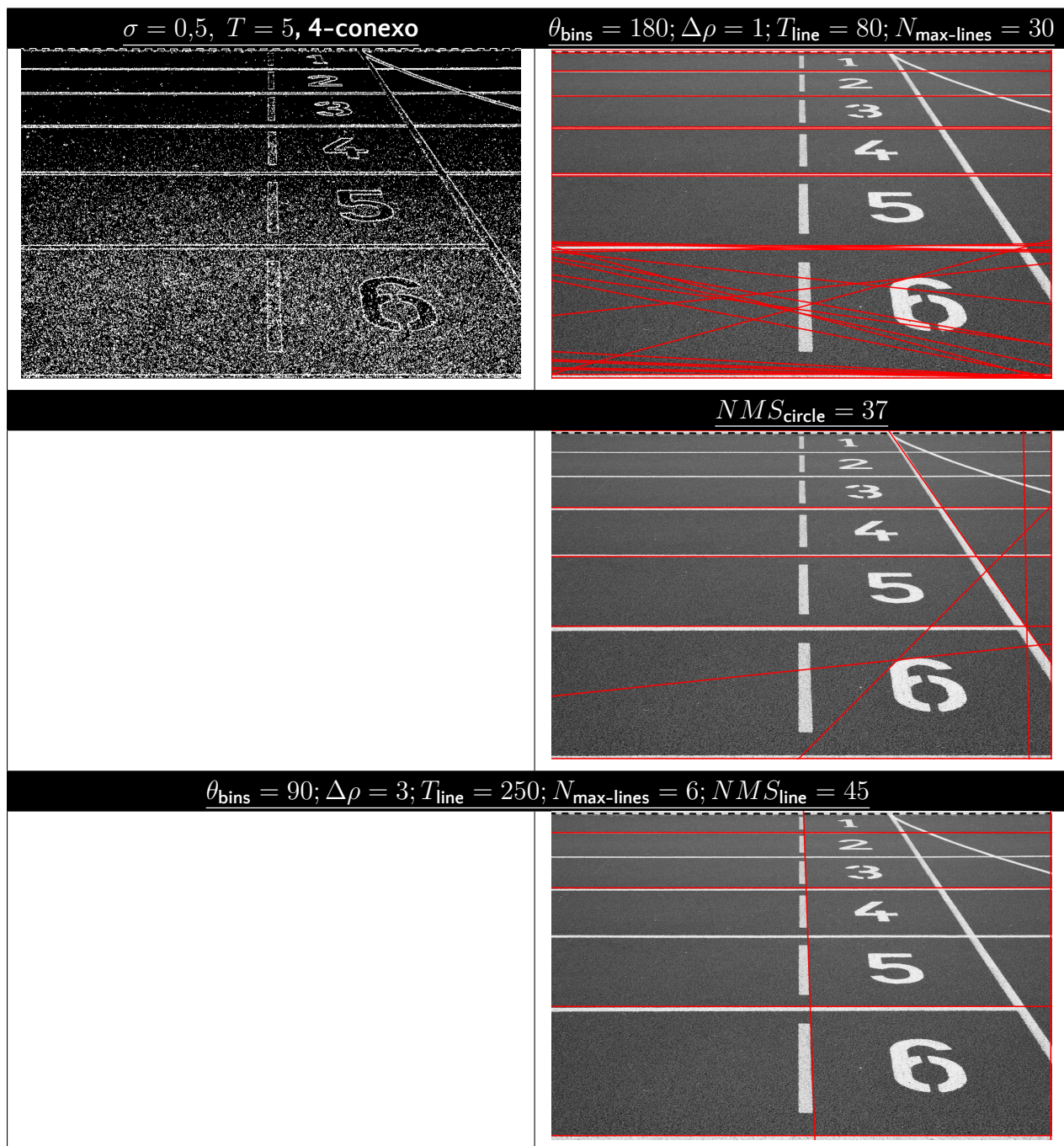


$N_{\text{max-lines}} = 10$



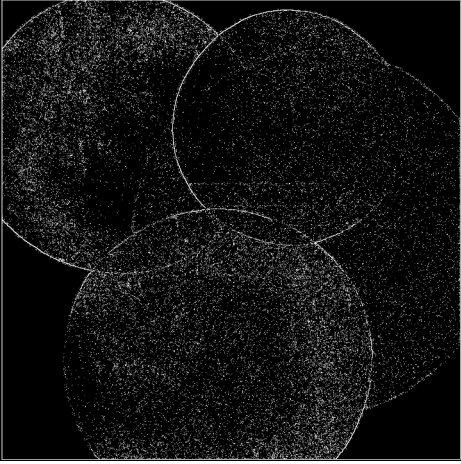
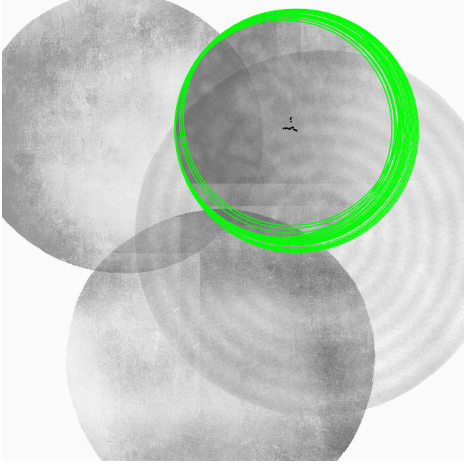
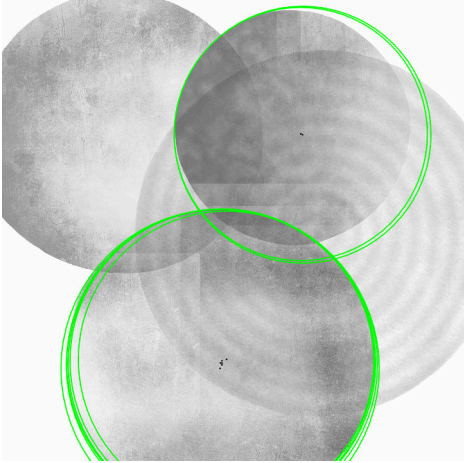
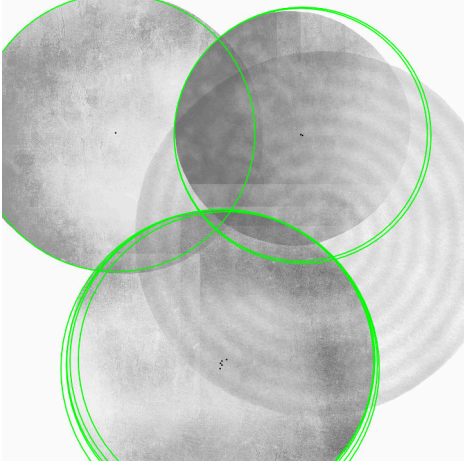
$N_{\text{max-lines}} = 12$; $NMS_{\text{line}} = 37$





Cuadro 16: Resultados de la transformada de Hough en la primera imagen.

Estos resultados muestran la robustez del método, ya que a pesar de obtener unos puntos de borde ruidosos, es posible obtener un buen resultado ajustando adecuadamente los parámetros de la transformada de Hough.

$\sigma = 1,8, T = 5, \text{8-conexo}$	
$r_{\min} = 200; r_{\max} = 800; \Delta r = 2;$ $\theta_{\text{circle-bins}} = 360; T_{\text{circle}} = 60;$ $NMS_{\text{circle}} = 500; N_{\text{max-circles}} = 200$	
	
$r_{\min} = 350; r_{\max} = 900; \Delta r = 5;$ $\theta_{\text{circle-bins}} = 360; T_{\text{circle}} = 60;$ $NMS_{\text{circle}} = 2; N_{\text{max-circles}} = 8$	
	
$r_{\min} = 350; r_{\max} = 900; \Delta r = 5;$ $\theta_{\text{circle-bins}} = 360; T_{\text{circle}} = 60;$ $NMS_{\text{circle}} = 500; N_{\text{max-circles}} = 8$	
	

Cuadro 17: Resultados de la transformada de Hough en la segunda imagen.

Contornos deformables

El método de los **contornos deformables** es una técnica de segmentación que busca ajustar una curva inicial a los bordes de un objeto en una imagen. La idea principal es que el contorno se **deforma dinámicamente** bajo la influencia de fuerzas internas y externas hasta coincidir con los límites de la región de interés.

Un contorno deformable se representa como una curva paramétrica:

$$v(s) = (x(s), y(s)), \quad s \in [0, 1]$$

donde $x(s)$ y $y(s)$ son las coordenadas de los puntos de la curva.

El ajuste del contorno se logra minimizando una **función de energía** compuesta por tres términos:

$$E = \sum_{i=1}^N (\alpha_i E_{\text{cont}} + \beta_i E_{\text{curv}} + \gamma_i E_{\text{imag}}) \quad \text{con } \alpha_i, \beta_i, \gamma_i \geq 0$$

- **Energía interna** (E_{cont}): controla la suavidad y continuidad del contorno. Evita que el contorno se rompa y que se formen ángulos bruscos.

$$E_{\text{cont}} = \|p_i - p_{i-1}\|^2$$

donde $p_i = (x_i, y_i)$.

- **Energía externa** (E_{curv}): controla la curvatura local del contorno, evitando cambios bruscos de dirección y favoreciendo formas suaves.

$$E_{\text{curv}} = \|p_{i-1} - 2p_i + p_{i+1}\|^2$$

donde $I(x, y)$ es la intensidad de la imagen.

- **Energía de atracción hacia los bordes** (E_{imag}): depende del gradiente de la imagen $I(x, y)$ y atrae el contorno hacia zonas con cambios de intensidad, es decir, los bordes.

$$E_{\text{imag}} = \|\nabla I\|^2$$

Los coeficientes α_i , β_i y γ_i son parámetros de ponderación que controlan la influencia relativa de cada término en la energía total:

- α_i : controla la **tensión** del contorno, favoreciendo la continuidad entre puntos consecutivos.
- β_i : regula la **rigidez** del contorno, penalizando curvaturas pronunciadas.
- γ_i : determina la **atracción hacia los bordes** de la imagen, guiando el contorno hacia zonas con alto gradiente.

Algoritmo, implementación y codificación

1. Se **inicializa el contorno** como una curva cerrada o abierta formada por puntos $p_i = (x_i, y_i)$, situada cerca del objeto que se desea segmentar. Esta inicialización puede hacerse manualmente o mediante heurísticas automáticas.
2. Se calcula la **energía total** del contorno:

$$E = \sum_{i=1}^N (\alpha_i E_{\text{cont}} + \beta_i E_{\text{curv}} + \gamma_i E_{\text{imag}})$$

donde cada término representa la continuidad, suavidad y atracción hacia los bordes, respectivamente.

3. Para cada punto del contorno, se evalúan las **fuerzas internas** (continuidad y curvatura) y las **fuerzas externas** (imagen), que determinan cómo debe moverse el punto para minimizar la energía.
4. Se **actualiza la posición** de cada punto p_i en función de las fuerzas calculadas. Este paso puede realizarse mediante métodos iterativos como el descenso de gradiente o técnicas de optimización numérica.
5. El proceso se repite hasta que el contorno **converge**, es decir, cuando las posiciones de los puntos dejan de cambiar significativamente o se alcanza un número máximo de iteraciones.
6. Finalmente, se **superpone el contorno** resultante sobre la imagen original para visualizar la región segmentada.

La codificación e implementación del método se ha realizado en el lenguaje **C++** con la ayuda de la librería *OpenCV*.

Resultados

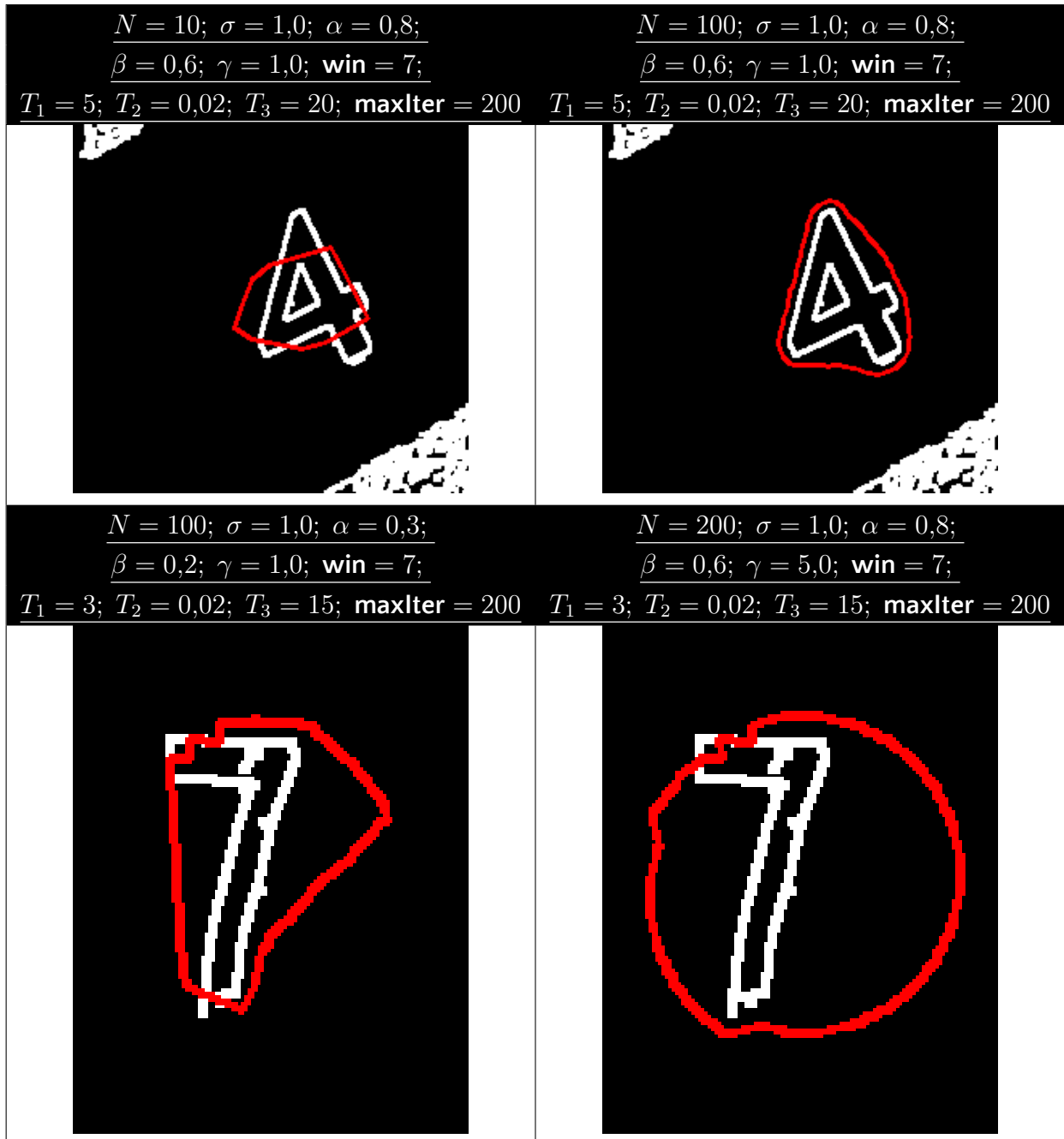
Se han obtenido los contornos de las figuras descritas por los dígitos 4 y 7. Para ello, se han empleado imágenes propias generadas para el conjunto de datos empleado en mi TFG [4]. Los resultados se presentan en el cuadro 18, donde se observa que un elevado número de puntos en la curva hace el contorno más sensible al ruido y flexible, pero lo empeora, mientras que un valor más bajo aumenta su rigidez. Por ejemplo, en las dos primeras imágenes se observa el efecto del número de puntos de la curva, mientras que en las dos últimas el número es demasiado elevado.

Además, valores bajos en el peso de continuidad ($\alpha = 0,3$) incrementan la deformabilidad del contorno, permitiendo que se adapte mejor a irregularidades y valores bajos en el peso de curvatura ($\beta = 0,2$) permiten que el contorno se ajuste a esquinas pronunciadas, como se aprecia en la tercera imagen de la tabla de resultados. En estas imágenes el valor del peso de imagen (γ) no tiene un gran efecto visual ya que son imágenes binarias y por tanto, la mayor parte son zonas planas donde el gradiente es cero y solo en el borde entre blanco y negro hay gradiente fuerte.

El resto de parámetros sirven para refinar el contorno, pero su variabilidad no se ha estudiado. El parámetro de vecindad (*win*) define el área de búsqueda local para cada punto; valores mayores aumentan la exploración pero también el coste computacional. Los umbrales T_1, T_2, T_3 regulan la convergencia:

- T_1 limita el número de puntos movidos por iteración, acelerando la parada si el contorno ya está estable.
- T_2 controla la curvatura mínima para reducir la rigidez en zonas de alta curvatura.
- T_3 establece la magnitud mínima del gradiente para que el contorno se adhiera a bordes significativos.

Finalmente, el número máximo de iteraciones (*maxlter*) asegura que el algoritmo se detenga incluso si no se alcanza la convergencia.



Cuadro 18: Resultados de la transformada de Hough en la segunda imagen.

Tema 6: operaciones morfológicas.

Operaciones Morfológicas

Las **operaciones morfológicas** son transformaciones aplicadas a imágenes que utilizan un **elemento estructurante**, B , para modificar su forma o estructura. Se emplean principalmente en el procesamiento de imágenes binarias, aunque también pueden extenderse a imágenes en escala de grises y de color. Su objetivo es resaltar, simplificar o analizar características geométricas de los objetos presentes en la imagen.

Las operaciones básicas son la dilatación y la erosión. De ambas surgen las operaciones derivadas principales, que son la apertura, el cierre y la detección de bordes. También existen otras operaciones como el llenado, el vaciado, el engrosado y el adelgazamiento.

Dilatación

La dilatación **expande las regiones blancas o de mayor intensidad** de la imagen, por lo que agranda los objetos y rellena huecos pequeños. En imágenes binarias se obtiene mediante la suma de *Minkowski*:

$$X \oplus B = \{x + b \mid x \in X, b \in B\}$$

Para el resto de imágenes, cada píxel de salida toma el valor máximo de sus vecinos definidos por el elemento estructurante:

$$(f \oplus B)(x, y) = \max_{(s, t) \in B} f(x - s, y - t)$$

El valor máximo de sus vecinos se conoce como máximo local de intensidad. Por ejemplo, si B es la matriz identidad 5×5 , se dilata al alrededor de cada píxel en los 5×5 píxeles vecinos con el máximo local.

Erosión

La erosión **reduce las regiones blancas o de mayor intensidad** de la imagen, por lo que adelgaza los objetos y elimina ruido aislado. En imágenes binarias se define como:

$$X \ominus B = \{d \in E^2 \mid d + b \in X \text{ para todo } b \in B\}$$

donde B_x es el elemento estructurante trasladado a la posición x . Para el resto de imágenes, cada píxel de salida toma el valor mínimo de sus vecinos definidos por el elemento estructurante:

$$(f \ominus B)(x, y) = \min_{(s, t) \in B} f(x + s, y + t)$$

El valor mínimo de sus vecinos se conoce como mínimo local de intensidad. Intuitivamente y en imágenes binarias, la erosión marca cada píxel con el mínimo local si alrededor del píxel en estudio contiene la estructura de B . Por tanto, toda la estructura queda marcada como 0 y el píxel central como 1, lo que indica que ahí se encontraba dicha estructura.

Apertura

La apertura **suaviza contornos, elimina objetos pequeños y ruido**, pero conserva la forma de los objetos grandes. Es la combinación de erosión seguida de dilatación:

$$f \circ B = (f \ominus B) \oplus B$$

Cierre

El cierre **rellena huecos pequeños y conecta regiones cercanas**, manteniendo la forma de los objetos principales. Es la combinación de dilatación seguida de erosión:

$$f \bullet B = (f \oplus B) \ominus B$$

Detección de Bordes

Los bordes morfológicos **resaltan los contornos de los objetos**. Se obtienen como la diferencia entre la dilatación y la erosión:

$$\text{Borde}(f) = (f \oplus B) - (f \ominus B)$$

Llenado

El llenado consiste en **completar regiones internas de los objetos**. Se puede realizar mediante operaciones iterativas de dilatación restringida al fondo:

$$\text{Llen}(X) = \lim_{k \rightarrow \infty} X_k, \quad X_k = (X_{k-1} \oplus B) \cap X^c$$

donde X es la imagen original y X^c su complemento. El proceso se repite hasta que $X_k = X_{k-1}$. Este proceso tiene una complicada implementación y alto coste computacional, por lo que se implementa mediante la siguiente aproximación: binarizar la imagen de bordes vaciada, aplicar un cierre para cerrar contornos y ejecutar la función *floodFill* de *OpenCV* para rellenar los huecos. Será necesario invertir la imagen para rellenar desde el exterior, es decir, posición (0, 0) de la imagen, de lo contrario necesitamos saber la posición de un píxel que tenga que ser rellando para que pueda ser utilizado como semilla.

Vaciado

El vaciado **elimina regiones interiores** de los objetos dejando solo sus contornos. Se define como:

$$\text{Vac}(X) = X - (X \ominus B)$$

Engrosado

El engrosado **incrementa el grosor** de los objetos sin alterar su forma global. Se implementa como una dilatación restringida:

$$\text{Engrosado}(X) = X \cup (X \oplus B)$$

Adelgazamiento

El adelgazamiento **reduce el grosor** de los objetos. Normalmente el adelgazamiento se aplica iterativamente hasta que los objetos alcanzan un grosor unitario. Se define como:

$$\text{Adelgazamiento}(X) = X \cap (X \ominus B)^c$$

Elemento estructurante B

Puede ser rectangular o completo, en forma de cruz, circular, en diagonal positiva o negativa, vertical u horizontal. Sea 5 su tamaño, se definen de la siguiente manera:

$$\begin{array}{ccc} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix} & \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix} & \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix} \\ \text{Rectangular} & \text{Cruz} & \text{Circular} \\ \\ \begin{bmatrix} 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \end{bmatrix} & \begin{bmatrix} 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \end{bmatrix} & \begin{bmatrix} 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \end{bmatrix} \\ \text{Diagonal positiva} & \text{Diagonal negativa} & \text{Vertical} \\ \\ & \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} & \\ & \text{Horizontal} \end{array}$$

Implementación

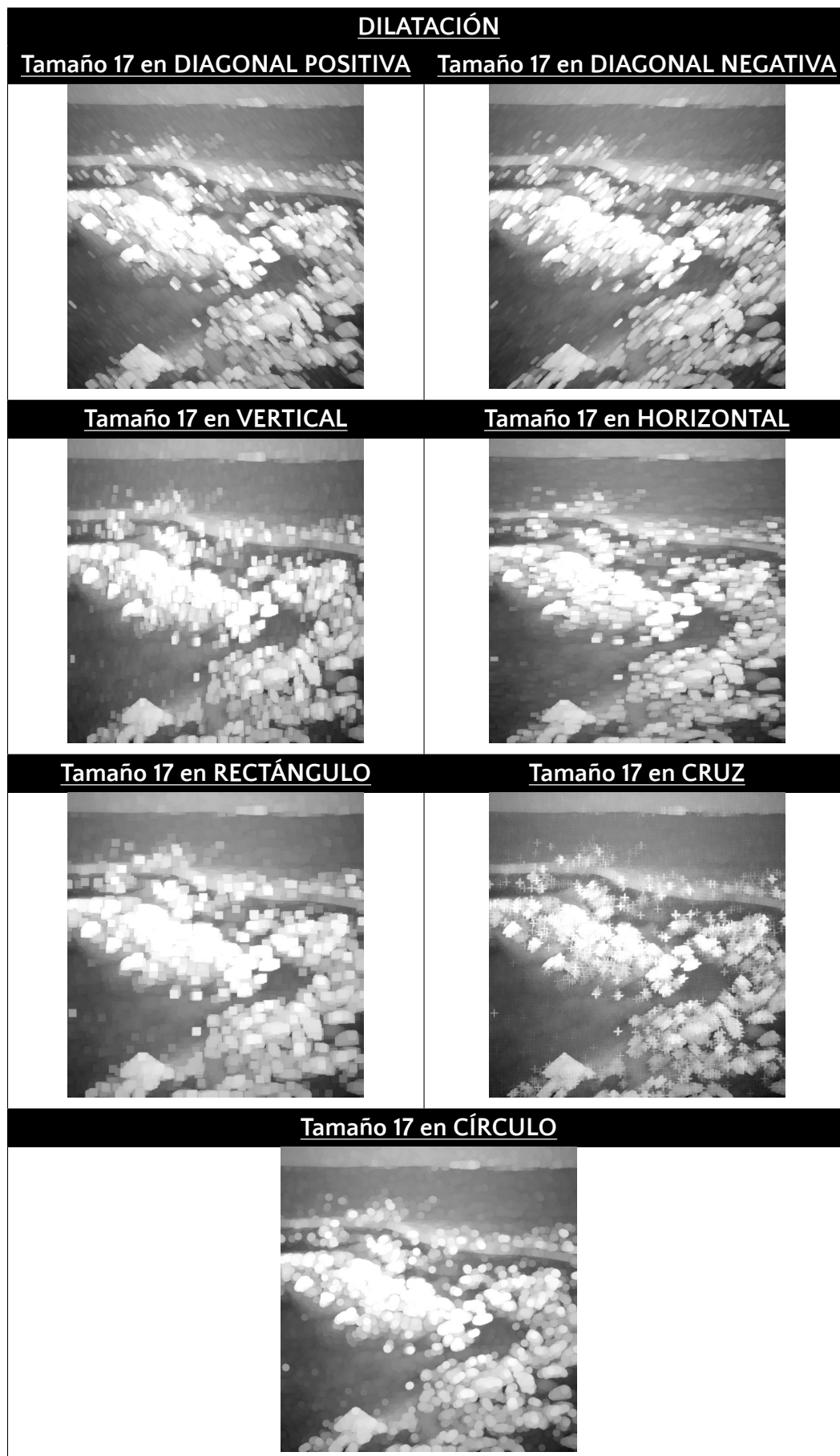
Estas operaciones se implementan fácilmente en **OpenCV** mediante funciones como `cv::dilate`, `cv::erode` y `cv::morphologyEx`. Sin embargo, del mismo modo que en implementaciones anteriores, se ha decidido implementar manualmente los métodos. Se ha realizado en el lenguaje de programación **C++**.

Resultados

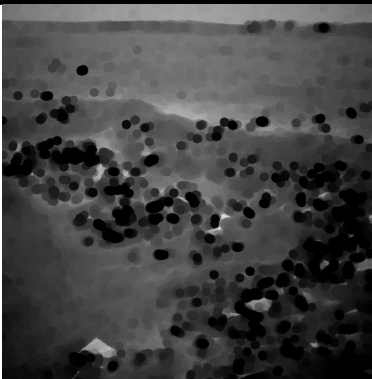
Se han aplicado las operaciones a la imagen proporcionada en los ejemplos de *Matlab*, figura 34, con diferentes tamaño y formas del elemento estructurante. Se han utilizado elementos estructurantes grandes para observar fácilmente el impacto de su forma. Los resultados se muestran entre los cuadros 19 y 27.



Figura 34: Imagen original sobre la que se han aplicado las operaciones morfológicas.



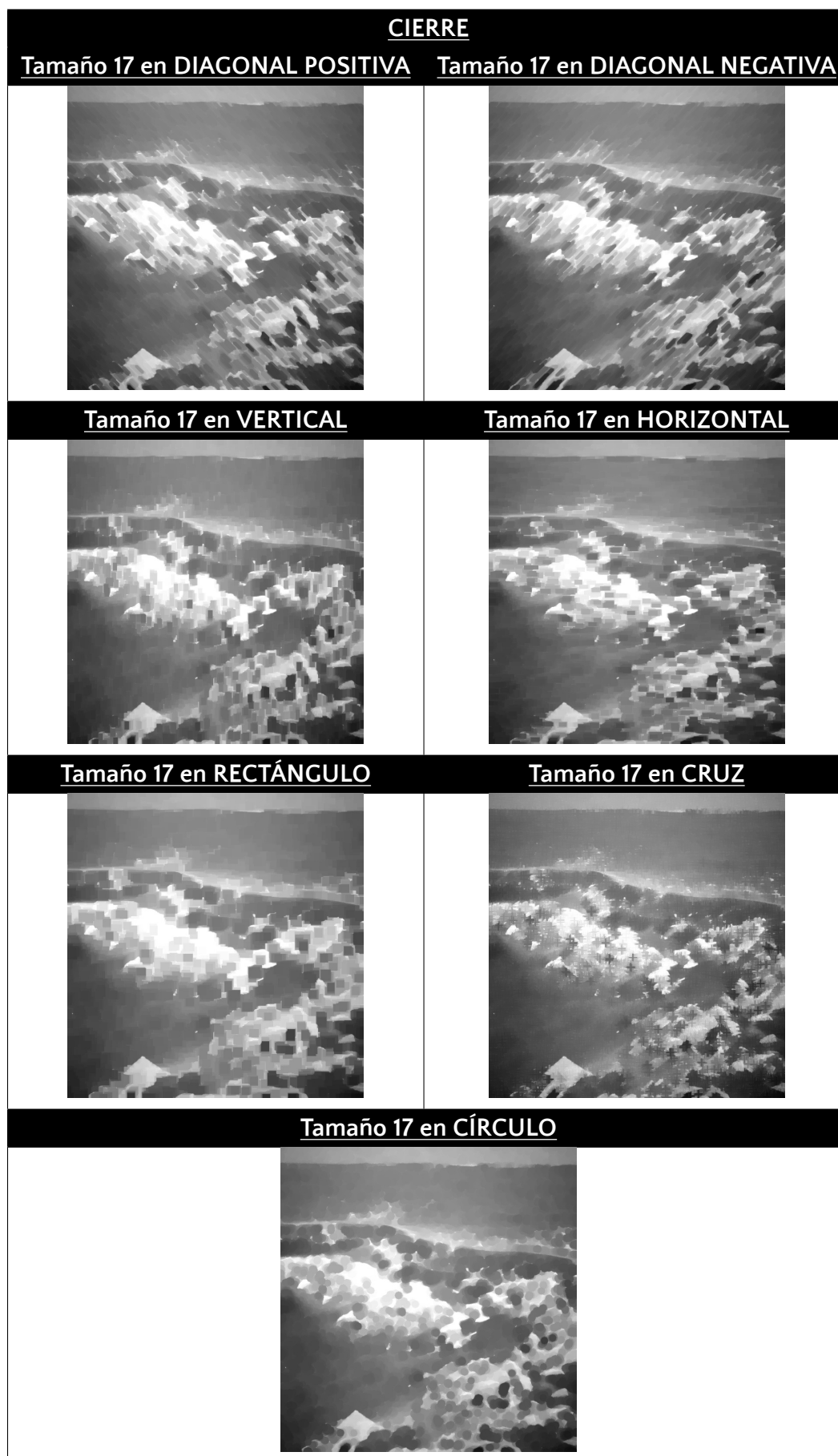
Cuadro 19: Resultados de la dilatación.

EROSIÓN	
Tamaño 17 en DIAGONAL POSITIVA	Tamaño 17 en DIAGONAL NEGATIVA
	
Tamaño 17 en VERTICAL	Tamaño 17 en HORIZONTAL
	
Tamaño 17 en RECTÁNGULO	Tamaño 17 en CRUZ
	
Tamaño 17 en CÍRCULO	
	

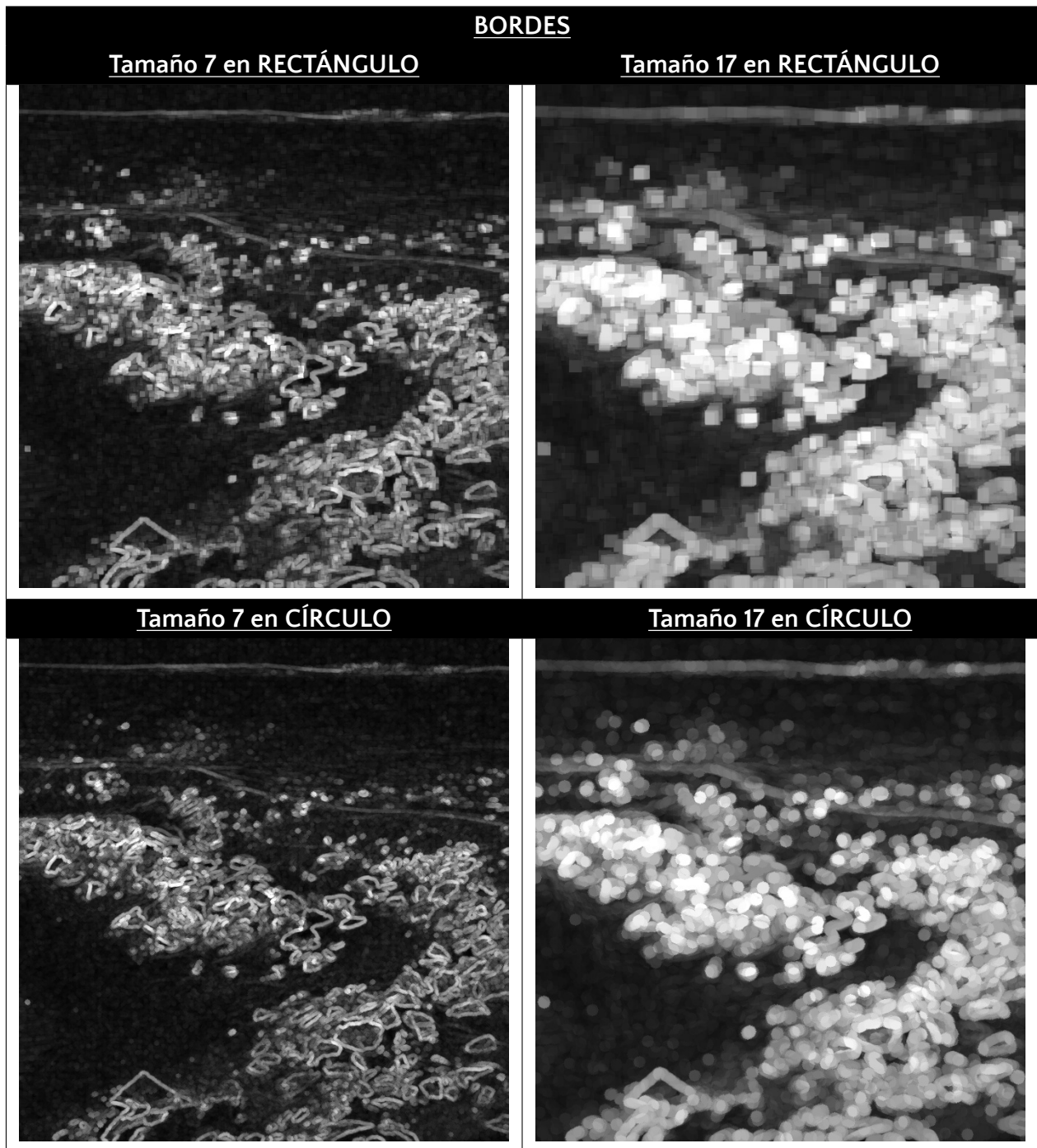
Cuadro 20: Resultados de la erosión.

<u>APERTURA</u>	
<u>Tamaño 17 en DIAGONAL POSITIVA</u>	<u>Tamaño 17 en DIAGONAL NEGATIVA</u>
	
<u>Tamaño 17 en VERTICAL</u>	<u>Tamaño 17 en HORIZONTAL</u>
	
<u>Tamaño 17 en RECTÁNGULO</u>	<u>Tamaño 17 en CRUZ</u>
	
<u>Tamaño 17 en CÍRCULO</u>	
	

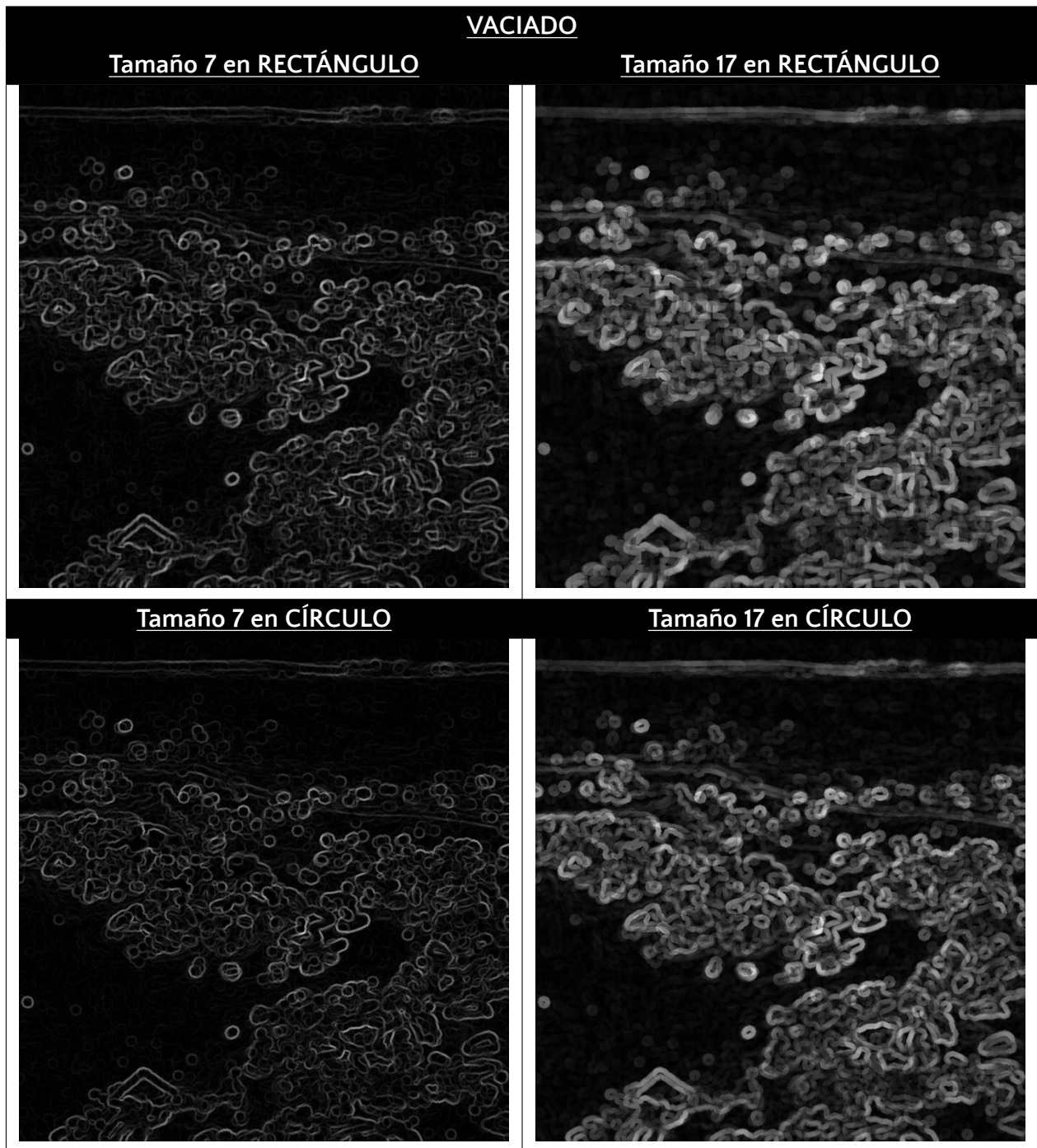
Cuadro 21: Resultados de la apertura.



Cuadro 22: Resultados del cierre.



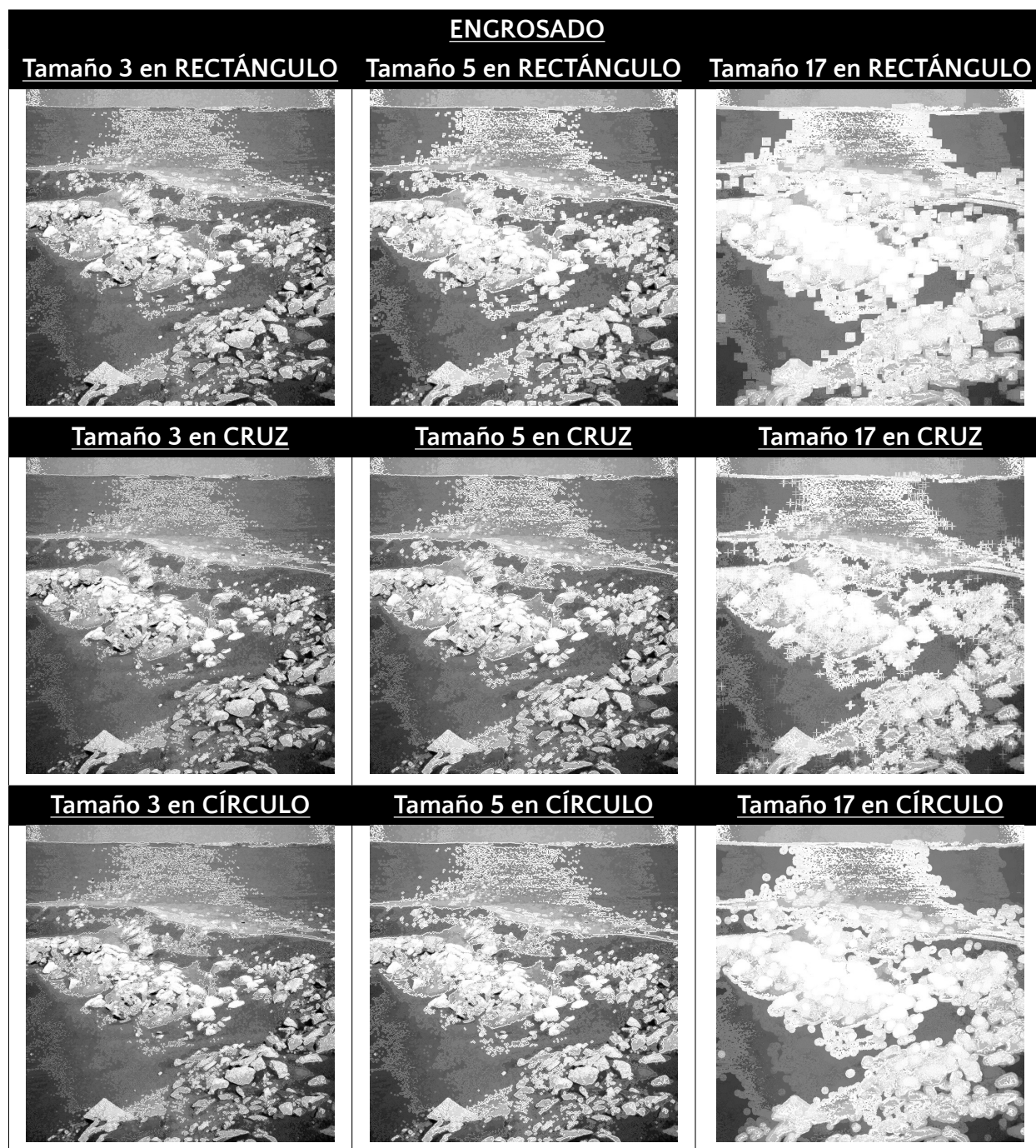
Cuadro 23: Resultados de la operación de bordes.



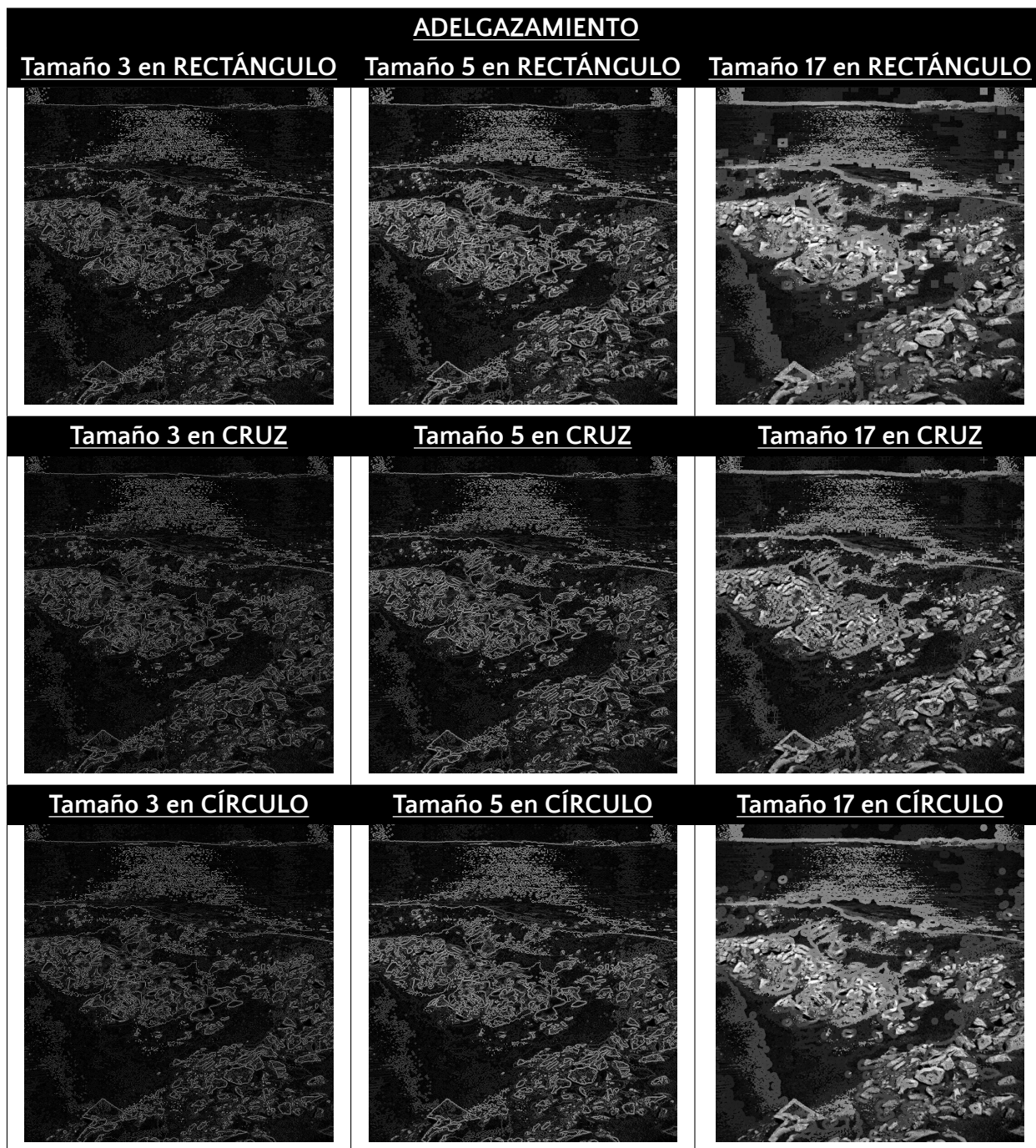
Cuadro 24: Resultados de la operación de vaciado sobre los bordes extraídos en rectángulo y círculo con un tamaño de 7 y 17.

LLENADO	
Tamaño 7 en RECTÁNGULO, $T = 50$	Tamaño 7 en RECTÁNGULO, $T = 80$
Tamaño 17 en RECTÁNGULO, $T = 50$	Tamaño 17 en RECTÁNGULO, $T = 80$
Tamaño 7 en CÍRCULO, $T = 50$	Tamaño 7 en CÍRCULO, $T = 80$
Tamaño 17 en CÍRCULO, $T = 50$	Tamaño 17 en CÍRCULO, $T = 80$

Cuadro 25: Resultados de la operación de llenado con distintos tamaños y formas del elemento estructurante y valor de umbral de binarización.



Cuadro 26: Resultados del engrosado.



Cuadro 27: Resultados del adelgazamiento.



Referencias adicionales al material del Campus Virtual

- [1] GeeksforGeeks. *Principal Component Analysis (PCA)*. <https://www.geeksforgeeks.org/data-analysis/principal-component-analysis-pca/>. Jul. de 2025.
- [2] G. Bradski. «The OpenCV Library». En: *Dr. Dobb's Journal of Software Tools* (2000). Último acceso: 19 de noviembre de 2025. URL: <https://opencv.org/>.
- [3] *Eigen: C++ Template Library for Linear Algebra*. Último acceso: 21 de noviembre de 2025. URL: <https://eigen.tuxfamily.org/>.
- [4] Javier Arambarri Calvo. *Desarrollo de algoritmos de percepción y control en ROS2 para robótica móvil*. 2025. URL: <https://github.com/arambarricalvoj/percepcion-control-ros2-tfg>.